

Chapter 8. Implementing Application Protocols

This chapter presents an overview of common Internet application protocols, and provides code examples of three protocol implementations under Java. Here we use the techniques covered in [Chapter 6](#) to create client and server software using sockets supporting popular Internet protocols.

8.1 Overview

An application protocol facilitates communication between applications, using the services of lower-level Open Systems Interconnection (OSI) model layers (for example, network and physical layers). When you check your e-mail, browse a Web site, play games, or download files over the Internet, the software you run is using an application protocol for communication.

For applications to interoperate, the implementation of application protocols must be precise. You can't have one application speaking one way and another application unable to understand or interpret the message—otherwise the many thousands of software applications running on the many millions of computers in existence couldn't get along. This is not to say that every implementation will support every feature of a protocol, or that they will be implemented in the same way, in the same language, or on the same machine. However, protocol implementations must, at least outwardly, behave in the same way, and when they can't fulfill a request or support a feature, they must communicate that effectively, using a commonly understood process.

The semantics of a protocol are laid out in a protocol specification document. This allows developers to implement the same protocol on a variety of platforms and languages and have them communicate successfully.

8.2 Application Protocol Specifications

Early on in the history of the Internet, a system was designed that made it easy to distribute new and updated protocol specifications to protocol implementers. The design of a network protocol tends to be an evolutionary process involving many contributors, with initial drafts being made available to the public and rewrites undertaken. When the protocol nears completion and is ready for applications to implement it, it is published as a Request For Comment (RFC) document.

Most of the popular Internet protocols in use today are published as RFCs. Each RFC details a single protocol or idea about the Internet, and is assigned a number for identification (e.g., RFC 1945 for HTTP/1.0). These documents are extremely detailed and contain all the information required for implementing the protocol. Not every RFC covers a protocol, however—some are general overviews, notes on Internet architecture, or suggestions for revisions to existing protocols.

NOTE



There's even the occasional attempt at humor in RFC documents—for those with an interest in all things Java, both the language and the beverage, you might want to take a look at RFC 2324, the Hyper Text Coffee Pot Control Protocol.

8.2.1 Finding RFC Documents

RFC documents are freely available and include a text-based version for ease of reading; they can be accessed using a Web browser. Though RFCs are mirrored across many hosts on the Internet, the original and most up-to-date source is the RFC Editor, operated by the Internet Society. It can be found at <http://www.rfc-editor.org/rfc.html>.

The RFC Editor Web site allows users to search for RFCs by number or by using keywords, authors, or titles. For example, if you wanted to look up RFC 2324, you'd search by number; entering "2324" would return the specification for the coffee pot control protocol mentioned above. If, on the other hand, you knew the name of the protocol but not its number, you'd search by title or keyword. For example, to search for the FTP you could enter the title "File Transfer Protocol." This query returns several RFCs, as some earlier specifications cover modifications to the protocol, or earlier versions of it. The higher the number, the more recent the RFC publication is. By running this query, you can quickly locate RFC 765, which contains the information needed to write compatible FTP clients or servers.

Alternatively, you can access RFCs by using an FTP client and connecting to the site <ftp.rfc-editor.org>. RFCs can be found under the /in-notes directory. By using a Web search engine, you can also find mirrors of RFC archives, some of which have converted the specifications to HTML format for easy reading.

8.3 Application Protocol Implementation

The most enjoyable part of learning network programming is putting the network theory that you've learned into practice, by writing real-life applications that interact with other Internet services or clients. Earlier in this book we covered the theory; now it's time to write some code. In the following sections, we'll examine and code three protocols—SMTP, POP3, and HTTP.

8.3.1 SMTP Client Implementation

The Simple Mail Transfer Protocol is used to send messages of various types between users over a TCP/IP network. It should be noted that this protocol assumes that some other method is used to actually read the messages, thus allowing a more stable, flexible, and robust global e-mail system. By separating delivering messages from reading, things are made much simpler.

Together, we'll write a basic SMTP client that allows the user to send a text message to a specific e-mail address. If you require something more elaborate, such as multiple senders or attachments, you could modify the code yourself as a programming exercise, or use the JavaMail API covered in [Chapter 13](#); it provides prewritten support for advanced mail features. The client written here offers a good example of networking, while minimizing such supporting code as that for a user interface. For this reason, simple text-based input is used, and the commands sent to the server are displayed to help the reader understand how the protocol works.

Code for SMTPClientDemo

```
import java.io.*;
import java.net.*;
import java.util.*;

// Chapter 8, Listing 1
```

```

public class SMTPClientDemo
{
    protected int port = 25;
    protected String hostname = "localhost";
    protected String from = "";
    protected String to = "";
    protected String subject = "";
    protected String body = "";
    protected Socket socket;
    protected BufferedReader br;
    protected PrintWriter pw;

    // Constructs a new instance of the SMTP Client
    public SMTPClientDemo() throws Exception
    {
        try
        {
            getInput();
            sendEmail();
        }
        catch (Exception e)
        {
            System.out.println ("Error sending message - " + e);
        }
    }

    public static void main(String[] args) throws Exception
    {
        // Start the SMTP client, so it can send messages
        SMTPClientDemo client = new SMTPClientDemo();
    }

    // Check the SMTP response code for an error message
    protected int readResponseCode() throws Exception
    {
        String line = br.readLine();
        System.out.println("< "+line);
        line = line.substring(0,line.indexOf(" "));
        return Integer.parseInt(line);
    }

    // Write a protocol message both to the network socket and to
    // the screen
    protected void writeMsg(String msg) throws Exception
    {
        pw.println(msg);
        pw.flush();
        System.out.println("> "+msg);
    }

    // Close all readers, streams and sockets
    protected void closeConnection() throws Exception
    {
        pw.flush();
        pw.close();
        br.close();
        socket.close();
    }

    // Send the QUIT protocol message, and terminate connection
    protected void sendQuit() throws Exception

```

```

{
    System.out.println("Sending QUIT");
    writeMsg("QUIT");
    readResponseCode();

    System.out.println("Closing Connection");
    closeConnection();
}

// Send an email message via SMTP, adhering to the protocol
// known as RFC 2821
protected void sendEmail() throws Exception
{
    System.out.println("Sending message now: Debug below");
    System.out.println("-----");

    System.out.println("Opening Socket");
    socket = new Socket(this.hostname,this.port);

    System.out.println("Creating Reader & Writer");
    br = new BufferedReader(new
        InputStreamReader(socket.getInputStream()));
    pw = new PrintWriter(new
        OutputStreamWriter(socket.getOutputStream()));

    System.out.println("Reading first line");
    int code = readResponseCode();
    if(code != 220) {
        socket.close();
        throw new Exception("Invalid SMTP Server");
    }

    System.out.println("Sending helo command");
    writeMsg("HELO "+InetAddress.getLocalHost().getHostName());
    code = readResponseCode();
    if(code != 250)
    {
        sendQuit();
        throw new Exception("Invalid SMTP Server");
    }

    System.out.println("Sending mail from command");
    writeMsg("MAIL FROM:<"+this.from+">");
    code = readResponseCode();
    if(code != 250)
    {
        sendQuit();
        throw new Exception("Invalid from address");
    }

    System.out.println("Sending rcpt to command");
    writeMsg("RCPT TO:<"+this.to+">");
    code = readResponseCode();
    if(code != 250)
    {
        sendQuit();
        throw new Exception("Invalid to address");
    }

    System.out.println("Sending data command");
    writeMsg("DATA");
}

```

```

code = readResponseCode();
if(code != 354)
{
    sendQuit();
    throw new Exception("Data entry not accepted");
}

System.out.println("Sending message");
writeMsg("Subject: "+this.subject);
writeMsg("To: "+this.to);
writeMsg("From: "+this.from);
writeMsg("");
writeMsg(body);
code = readResponseCode();
sendQuit();
if(code != 250)
    throw new Exception("Message may not have been sent correctly");
else
    System.out.println("Message sent");
}

// Obtain input from the user
protected void getInput() throws Exception
{
    // Read input from user console
    String data=null;
    BufferedReader br = new BufferedReader (
        new InputStreamReader(System.in));

    // Request hostname for SMTP server
    System.out.print("Please enter SMTP server hostname: ");
    data = br.readLine();

    if (data == null || data.equals("")) hostname="localhost";
    else
        hostname=data;

    // Request the sender's email address
    System.out.print("Please enter FROM email address: ");
    data = br.readLine();
    from = data;

    // Request the recipient's email address
    System.out.print("Please enter TO email address :");
    data = br.readLine();
    if(!(data == null || data.equals(""))))
        to=data;

    System.out.print("Please enter subject: ");
    data = br.readLine();
    subject=data;

    System.out.println(
        "Please enter plain-text message ('.' character on
        a blank line signals end of message):");
    StringBuffer buffer = new StringBuffer();

    // Read until user enters a . on a blank line
    String line = br.readLine();
    while(line != null)
    {

```

```

        // Check for a '.', and only a '.', on a line
        if (line.equalsIgnoreCase("."))
        {
            break;
        }
        buffer.append(line);
        buffer.append("\n");
        line = br.readLine();
    }
    buffer.append("\n");
    body = buffer.toString();
}
}

```

How SMTPClientDemo Works

As can be seen here, the Simple Mail Transfer Protocol is quite straightforward, consisting of a single connection to a mail server using a TCP socket, followed by a series of short protocol commands that specify the details of the e-mail to be sent, as described in RFC 2821. While many network applications will require multiple threads of execution, as a general rule simple clients such as this one do not. This example merely asks for input from the user and then sends the message.

To simplify understanding of this code, the networking code has been separated from the non-networking code (which consists mainly of code for obtaining input from the user).

The basic skeleton of the application is as follows:

```

public class SMTPClientDemo
{
    public static void main(String[] args) throws Exception
    {
        SMTPClientDemo client = new SMTPClientDemo ();
    }
    // Constructs a new instance of the SMTP Client
    public SMTPClientDemo () throws Exception;

    // Send an email message via SMTP, adhering to the protocol
    // known as RFC 2821
    protected void sendEmail() throws Exception;

    // Check the SMTP response code for an error message
    protected int readResponseCode() throws Exception;

    // Write a protocol message both to the network socket and to
    // the screen
    protected void writeMsg(String msg) throws Exception;

    // Close all readers, streams and sockets
    protected void closeConnection() throws Exception;

    // Send the QUIT protocol message, and terminate connection
    protected void sendQuit() throws Exception;

    // Obtain input from the user
    protected void getInput() throws Exception;
}

```

We'll cover the e-mail-specific code, as the remainder of the application is fairly straightforward Java coding. Our class stores the message and network details inside protected variables (rather than private ones, so readers can create subclasses that access these variables if required). These variables are:

- `String hostname`
- `int port` (set to 25, the default for SMTP)
- `String from`
- `String to`
- `String subject`
- `String body`
- `java.net.Socket socket`
- `BufferedReader br`
- `PrintWriter pw`

The `hostname`, `port`, `from`, `to`, `subject`, and `body` variables are fairly self-explanatory—they are required for locating the mail server to send an SMTP message, and for the e-mail addressing and content details. The `socket` is used to communicate with the remote TCP server that will relay our mail for us, and the `reader/writer` objects are used for reading and sending SMTP messages.

When the application is run, the `main()` method is executed by the Java Virtual Machine. This in turn creates an instance of our `SMTPClientDemo` application, which requests input data from the user and attempts to send an e-mail message. The process has been fairly simple, so far. Now let's turn to the application protocol code.

Contained inside the `sendEmail()` method is the heart of the program. This is where we connect a local TCP socket to a remote SMTP server, and send across protocol requests. While it's certainly possible to group this process into one single method, for convenience and readability, some of the workload has been distributed across several helper methods that decode SMTP responses, send protocol messages, and terminate connections.

The first thing the `sendEmail()` method does is open a socket connection to the mail server located at port n , where n is represented by the member variable `port`. The SMTP normally uses port 25, but if your network/ISP uses a nonstandard port number for this service, you could modify the default value of this to the required port number. If an error occurs, an exception is thrown and caught by our error handler; otherwise a successful connection to the server is established.

Once connected, we obtain input streams and output streams for the socket, and connect these to readers and writers for convenience. According to the SMTP specification, upon connecting, the server will send a response code in greeting, so the application checks for a valid message with the aid of helper method `readResponseCode()`. This helps identify that we really are talking to an SMTP service, and not some other service using port 25.

The code for determining the response code is wrapped inside a helper method, called `readResponseCode()`. It takes no parameters, and returns an `int` representing the code number. It reads a line of text from the buffered reader that communicates with the SMTP server, outputs the result to the screen for illustrative purposes, and then strips off everything after the first space. Finally, it converts the result to an integer and passes it back, giving us the SMTP response code.

```
// Check the SMTP response code for an error message
protected int readResponseCode() throws Exception
{
    String line = br.readLine();
    System.out.println("< " + line);
}
```

```

    line = line.substring(0, line.indexOf(" "));
    return Integer.parseInt(line);
}

```

When we need to send a message back to the SMTP server, we use the helper method `writeMsg(String)` which displays the message to the screen for illustrative purposes and then sends it (via our `PrintWriter`) to the server.

```

// Write a protocol message both to the network socket and to
// the screen
protected void writeMsg(String msg) throws Exception
{
    pw.println(msg);
    pw.flush();
    System.out.println("> "+msg);
}

```

Using these helper methods, the `sendEmail()` method can send and receive SMTP messages. It checks, upon connecting, that the server has sent an OK message, which is represented by 220.

```

int code = readResponseCode();
if(code != 220) {
    socket.close();
    throw new Exception("Invalid SMTP Server");
}

```

The next step in SMTP is to send an identification message, telling the SMTP server who we really are. We send the hostname identification command, "HELO." The format is "HELO" followed by a space and the local hostname. The correct response is 250, which signals OK.

```

writeMsg("HELO "+InetAddress.getLocalHost().getHostName());
code = readResponseCode();
if(code != 250)
{
    sendQuit();
    throw new Exception("Invalid SMTP Server");
}

```

If the server accepts the identification, the client is free to send a message. Each message has certain key aspects—a "From" address, one or more "To" addresses, a subject line, and a message body (the actual text of the message). The SMTP only requires fields that deal directly with delivery of messages, though other fields can be placed in the message body. For example, it is commonly accepted to put the subject line as the first line, with "Subject:" in front of it. You can include as many other optional fields as you like, but remember that not every mail client will support these fields. Furthermore, you should also repeat the "To" and "From" fields in your message body.

Setting the sender and recipient addressing information under SMTP is fairly straightforward. Two commands are used, the "MAIL FROM:" and "RCPT TO:"

```

writeMsg("MAIL FROM:<"+this.from+">");

// Check response from server
// .....

writeMsg("RCPT TO:<"+this.to+">");
// Check response from server
// .....

```


In this example, only one recipient is supported, but SMTP can handle multiple recipients. You should note that well-configured servers normally place a limit on the number of possible recipients, to prevent them from being used in spamming campaigns, and that some SMTP servers will reject messages if the sender is not part of their local or dial-up network.

Once the e-mail addresses are set, we can send the data to the server. Our client sends a simple text message, unencumbered by attachments. The first step to send the data is to signal that we are ready to send a message body, by issuing a "DATA" command. The valid response code for this is 354.

```
System.out.println("Sending data command");
writeMsg("DATA");
code = readResponseCode();
if(code != 354)
{
    sendQuit();
    throw new Exception("Data entry not accepted");
}
```

Now the client sends the message body. This must include relevant header fields, such as "To," "From," "Subject," and any other fields you may choose to add. Once the headers are complete, a blank line is sent, indicating that the message text will follow. After outputting the text, the client sends the message, which is then terminated by a period, then a carriage return/line-feed (which the user enters during data input). This tells the server that the message is complete.

```
writeMsg("Subject:"+this.subject);
writeMsg("");
writeMsg(body);
code = readResponseCode();
sendQuit();
if(code != 250)
    throw new Exception("Message may not have been sent correctly");
else
    System.out.println("Message sent");
```

Finally, the "QUIT" command is sent, and the connection is terminated by invoking the `sendQuit()` method. Our transaction with the SMTP server is complete, and the message is on its way.

```
// Send the QUIT protocol message, and terminate connection
protected void sendQuit() throws Exception
{
    System.out.println("Sending QUIT");
    writeMsg("QUIT");
    readResponseCode();

    System.out.println("Closing Connection");
    closeConnection();
}
```

Running SMTPClientDemo

After compiling, the application can be run by typing:

```
java SMTPClientDemo
```

The application will request the following information:

- The name of a valid SMTP server (such as that used by your e-mail program)
- The "From" address of the sender (e.g., your e-mail address)
- The "To" address of the recipient (e.g., your e-mail address, so you know that it was delivered)
- The "Subject" of the message
- The message contents

8.3.2 POP3 Client Implementation

The previous example showed how to write a protocol implementation of SMTP to send a mail message. In order to read mail, you will need the Post Office Protocol version 3, or POP3 as it is commonly referred to. It is also referred to simply as POP, since the earlier Post Office Protocols are not used any more. This term is less confusing for general Internet users, who may wonder what happened to POP1 and POP2, which have been superseded.

When a person reads their e-mail, they are actually retrieving it from a mail server belonging to an Internet service provider or a local network (such as a corporate or private intranet). There are several major types of mail servers, and each type uses a different network protocol to access e-mail. The most common of these is POP, a protocol that provides a contrast with SMTP.

In examining the POP3 client, you'll find that the actual code required for a network implementation is minimal. Much of the actual application involves user input or output. The interface is limited to text I/O; the idea behind this client is to provide a good example of networking while minimizing the need for non-networking code that readers may already be familiar with, such as GUI design. The example will retrieve messages from a mailbox and display their contents, one after another, to the text console screen.

Code for Pop3ClientDemo

```
import java.io.*;
import java.net.*;
import java.util.*;

// Chapter 8, Listing 2
public class Pop3ClientDemo
{
    protected int port = 110;
    protected String hostname = "localhost";
    protected String username = "";
    protected String password = "";

    protected Socket socket;
    protected BufferedReader br;
    protected PrintWriter pw;

    // Constructs a new instance of the POP3 client
    public Pop3ClientDemo() throws Exception
    {
        try
        {
            // Get user input
            getInput();

            // Get mail messages
            displayEmails();
        }
        catch (Exception e)
        {
        }
    }
}
```

```

        System.err.println ("Error occured - details follow");
        e.printStackTrace();
        System.out.println(e.getMessage());
    }
}

// Returns TRUE if POP response indicates success, FALSE if
// failure
protected boolean responseIsOk() throws Exception
{
    String line = br.readLine();
    System.out.println("< "+line);
    return line.toUpperCase().startsWith("+OK");
}

// Reads a line from the POP server, and displays it to
// screen
protected String readLine(boolean debug) throws Exception
{
    String line = br.readLine();
    // Append a < character to indicate this is a server protocol
response
    if (debug)
        System.out.println("< "+line);
    else
        System.out.println(line);
    return line;
}

// Writes a line to the POP server, and displays it to the screen
protected void writeMsg(String msg) throws Exception
{
    pw.println(msg);
    pw.flush();
    System.out.println("> "+msg);
}

// Close all writers, streams and sockets
protected void closeConnection() throws Exception
{
    pw.flush();
    pw.close();
    br.close();
    socket.close();
}

// Send the QUIT command, and close connection
protected void sendQuit() throws Exception
{
    System.out.println("Sending QUIT");
    writeMsg("QUIT");
    readLine(true);

    System.out.println("Closing Connection");
    closeConnection();
}

// Display emails in a message
protected void displayEmails() throws Exception
{
    BufferedReader userInput = new BufferedReader( new

```

```

        InputStreamReader (System.in) );

System.out.println("Displaying mailbox with protocol commands
    " and responses below");
System.out.println("-----+
    "-----");

// Open a connection to POP3 server
System.out.println("Opening Socket");
socket = new Socket(this.hostname, this.port);
br = new BufferedReader(new
    InputStreamReader(socket.getInputStream()));
pw = new PrintWriter(new
    OutputStreamWriter(socket.getOutputStream()));

// If response from server is not okay
if(! responseIsOk())
{
    socket.close();
    throw new Exception("Invalid POP3 Server");
}

// Login by sending USER and PASS commands
System.out.println("Sending username");
writeMsg("USER "+this.username);
if(!responseIsOk())
{
    sendQuit();
    throw new Exception("Invalid username");
}

System.out.println("Sending password");
writeMsg("PASS "+this.password);
if(!responseIsOk())
{
    sendQuit();
    throw new Exception("Invalid password");
}

// Get mail count from server ....
System.out.println("Checking mail");
writeMsg("STAT" );

// ... and parse for number of messages
String line = readLine(true);
StringTokenizer tokens = new StringTokenizer(line, " ");
tokens.nextToken();
int messages = Integer.parseInt(tokens.nextToken());
int maxsize = Integer.parseInt(tokens.nextToken());

if (messages == 0)
{
    System.out.println ("There are no messages.");
    sendQuit();
    return;
}

System.out.println ("There are " + messages + " messages.");
System.out.println("Press enter to continue.");
userinput.readLine();

```

```

        for(int i = 1; i <= messages ; i++)
        {
            System.out.println("Retrieving message number "+i);
            writeMsg("RETR "+i);
            System.out.println("-----");
            line = readLine(false);
            while(line != null && !line.equals("."))
            {
                line = readLine(false);
            }
            System.out.println("-----");
            System.out.println("Press enter to continue." +
                "To stop, type Q then enter");
            String response = userInput.readLine();
            if (response.toUpperCase().startsWith("Q"))
                break;
        }
        sendQuit();
    }

    public static void main(String[] args) throws Exception
    {
        Pop3ClientDemo client = new Pop3ClientDemo();
    }

    // Read user input
    protected void getInput() throws Exception
    {
        String data=null;
        BufferedReader br = new BufferedReader
            (new InputStreamReader(System.in));
        System.out.print("Please enter POP3 server hostname:");
        data = br.readLine();
        if(data == null || data.equals("")) hostname= "localhost";
        else
            hostname=data;

        System.out.print("Please enter mailbox username:");
        data = br.readLine();
        if(!(data == null || data.equals(""))
            username=data;

        System.out.print("Please enter mailbox password:");
        data = br.readLine();
        if(!(data == null || data.equals(""))
            password=data;
    }
}

```

How Pop3ClientDemo Works

The application is structured similarly to the SMTP client, in that the `main()` method constructs a new instance of the class, which then performs all of the work. In fact, the chief difference is a call to the `displayEmails` method instead of to the `sendEmail` method. In addition, there are several helper methods that assist in conducting communication via POP3.

Like the SMTP application, several important variables are requested from the user. To retrieve e-mail messages, you need to know the hostname of the mail server, the username of the account, and the password for accessing it. These details are obtained by the `getInput()` method of the application, which uses simple text I/O to request details from the user. These are then stored in

member variables for later access. We also store the port number of the server (which is fixed to the default port of 110). If, for example, you needed to support nonstandard POP servers, you could modify this value.

```
public class Pop3ClientDemo
{
    protected int port = 110;
    protected String hostname = "localhost";
    protected String username = "";
    protected String password = "";

    // .....
}
```

The most important part of the application is the Post Office Protocol implementation. This is where we get to write real networking code. To make things clearer and more efficient, some of the protocol code is split into helper methods, which perform tasks such as processing a POP response to see that no error code was issued, and reading/writing protocol commands. The main work is done, however, in the `displayEmails()` method.

We start by creating a network socket to the POP server, using the `Socket` class. The next step is to obtain readers and writers connected to the socket stream, so that we can communicate with the server.

```
// Open a connection to POP3 server
System.out.println("Opening Socket");
socket = new Socket(this.hostname, this.port);

br = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
pw = new PrintWriter(new
    OutputStreamWriter(socket.getOutputStream()));
```

Upon establishing a connection, the server sends a POP response indicating that the server is ready for commands. If the client does not receive a valid POP response, either the server is malfunctioning or a non-POP server is operating on that port. For this reason, the client must always check the response code after opening a TCP connection. We use the `responseOk()` helper method to determine this, which returns a boolean value. Note the negation of the "if" statement—we close the connection only if the response is not okay (indicated by the "!" operator).

```
// If response from server is not okay
if(!responseIsOk())
{
    socket.close();
    throw new Exception("Invalid POP3 Server");
}
```

Now our application is ready to send POP commands. The first command that is sent is "USER," which identifies the user account that the client is trying to access. We send the command, along with the username, by using the helper method `writeMsg(String)`. This also outputs the command to the text console, so you can see how the protocol is working. We do the same then with the "PASS" command, which is used to authenticate the identity of the user.

```
// Login by sending USER and PASS commands
System.out.println("Sending username");
writeMsg("USER "+this.username);
if(!responseIsOk())
{
```

```

        sendQuit();
        throw new Exception("Invalid username");
    }

    writeMsg("PASS "+this.password);
    if(!responseIsOk())
    {
        sendQuit();
        throw new Exception("Invalid password");
    }
}

```

At this point, the application will be ready to send commands, unless user authentication failed, in which case the application will have terminated. Now we can request the number of mail messages available from the server. This is achieved by sending the "STAT" command, which returns the number of messages. We parse the response, looking for the message count, and then convert it to an `int` value.

```

writeMsg("STAT");

// ... and parse for number of messages

String line = readLine(true);
StringTokenizer tokens = new StringTokenizer(line, " ");
tokens.nextToken();
int messages = Integer.parseInt(tokens.nextToken());
int maxsize = Integer.parseInt(tokens.nextToken());

```

Armed with this information, we can determine if the mailbox is empty or if there are messages to be retrieved. If so, the application simply loops and displays the message contents. During each loop, the user is presented with the option of quitting in case the message count is too high.

```

System.out.println("There are " + messages + " messages.");
System.out.println("Press enter to continue.");
userinput.readLine();

for(int i = 1; i <= messages ; i++)
{
    System.out.println("Retrieving message number "+i);
    writeMsg("RETR "+i);
    System.out.println("-----");
    line = readLine(false);
    while(line != null && !line.equals("."))
    {
        line = readLine(false);
    }
    System.out.println("-----");
    System.out.println("Press enter to continue. To stop, type Q then enter");
    String response = userinput.readLine();
    if (response.toUpperCase().startsWith("Q"))
        break;
}
sendQuit();

```

Finally, we call the `sendQuit()` method, which shuts down the server connection. The application then quits and the task of reading mail using the Post Office Protocol is complete. As you can see, reading e-mail is a fairly simple task, perhaps even easier than sending it.

Running Pop3ClientDemo

Running the application, too, is quite manageable. After compiling, simply type:

```
java Pop3ClientDemo
```

You will be prompted for the hostname of your mail server (which, if you're not sure of, can be obtained by looking at your e-mail client's settings), the username, and then the password. For example, a user might type the following:

```
java PopClientDemo
Please enter POP3 server hostname: myserver.myisp.com
Please enter mailbox username: johndoe
Please enter mailbox password: javaduke
```

8.3.3 HTTP/1.0 Server Implementation

The HyperText Transfer Protocol (HTTP) originated as a means of sharing documents across the Internet. Since many documents are interrelated, the need to provide a link from one to the other was identified, but given the fact that many researchers from around the world were working independently, a single centralized document server was not the ideal method. By placing a hyperlink over a word (such as a scientific term) or a phrase, users would be able to jump instantly from one document to another, even though the documents could reside on servers located in other countries. Not only that, but hyperlinks could be made within the same document, such as to a glossary of terms. Published as RFC 1945, HTTP became one of the most quickly adopted protocols, and led to what we now know as the World Wide Web. Pioneered by Tim Berners-Lee and his colleagues at the CERN scientific laboratories as a way to share scientific information, it quickly spread to other parts of the academic world, and then to commercial and consumer markets.

The first, and most widely supported version, of HTTP is known as HTTP/1.0. This protocol supports a simple set of commands for retrieving resources from a Web server, such as HTML pages, images, documents, and other file types, as well as commands for posting information to the Web server so as to allow for the interactivity and customization of Web pages. This capability is particularly important for Web sites that support advanced features, such as user customization or shopping carts.

The latest version of the protocol is known as HTTP/1.1. It offers many improvements over the previous version, with a wider set of commands to support new features of use both to browser developers and to those who write server-side Web applications. However, older browsers and servers will not support all of these features, and indeed some are not yet in common use at all.

This next example demonstrates how to write a multi-threaded HTTP server that responds to requests from a Web browser, fetches files or Web pages, and sends them back to the user. For the purposes of this example, we will use version 1.0 of the HTTP protocol and support only the `GET` method, which is used for file retrieval. Other methods, such as `POST`, are useful when designing interactive server-side applications. However, for such uses it is advised that the reader consider using a fully-fledged commercial server, and Java servlets (discussed in later chapters).

Code for WebServerDemo

```
import java.io.*;
import java.net.*;
import java.util.*;
// Chapter 8, Listing 3
public class WebServerDemo
{
    // Directory of HTML pages and other files
```



```

protected String docroot;
// Port number of web server
protected int port;
// Socket for the web server
protected ServerSocket ss;

// Handler for a HTTP request
class Handler extends Thread
{
    protected Socket socket;
    protected PrintWriter pw;
    protected BufferedOutputStream bos;
    protected BufferedReader br;
    protected File docroot;

    public Handler(Socket _socket, String _docroot) throws Exception
    {
        socket=_socket;
        // Get the absolute directory of the filepath
        docroot=new File(_docroot).getCanonicalFile();
    }

    public void run()
    {
        try
        {
            // Prepare our readers and writers
            br = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            bos = new
BufferedOutputStream(socket.getOutputStream());
            pw = new PrintWriter(new OutputStreamWriter(bos));

            // Read HTTP request from user (hopefully GET /file..... )
            String line = br.readLine();

            // Shutdown any further input
            socket.shutdownInput();

            if(line == null)
            {
                socket.close();
                return;
            }
            if(line.toUpperCase().startsWith("GET"))
            {
                // Eliminate any trailing ? data, such as for a CGI
                // GET request
                StringTokenizer tokens = new StringTokenizer(line," ?");
                tokens.nextToken();
                String req = tokens.nextToken();

                // If a path character / or \ is not present, add it
                // to the document root
                // and then add the file request, to form a full
                // filename
                String name;
                if(req.startsWith("/") || req.startsWith("\\\\"))
                    name = this.docroot+req;
                else
                    name = this.docroot+File.separator+req;
            }
        }
    }
}

```

```

        // Get absolute file path
        File file = new File(name).getCanonicalFile();

        // Check to see if request doesn't start with our
        // document root ....

if(!file.getAbsolutePath().startsWith(this.docroot.getAbsolutePath()))
    {
        pw.println("HTTP/1.0 403 Forbidden");
        pw.println();
    }
    // ... if it's missing .....
    else if(!file.exists())
    {
        pw.println("HTTP/1.0 404 File Not Found");
        pw.println();
    }
    // ... if it can't be read for security reasons ....
    else if(!file.canRead())
    {
        pw.println("HTTP/1.0 403 Forbidden");
        pw.println();
    }
    // ... if its actually a directory, and not a file ....
    else if(file.isDirectory())
    {
        sendDir(bos,pw,file,req);
    }
    // ... or if it's really a file
    else
    {
        sendFile(bos, pw, file.getAbsolutePath());
    }
    }
    // If not a GET request, the server will not support it
    else
    {
        pw.println("HTTP/1.0 501 Not Implemented");
        pw.println();
    }

    pw.flush();
    bos.flush();
}
catch(Exception e)
{
    e.printStackTrace();
}
try
{
    socket.close();
}
catch(Exception e)
{
    e.printStackTrace();
}
}

protected void sendFile(BufferedOutputStream bos,
    PrintWriter pw, String filename) throws Exception

```

```

{
    try
    {
        java.io.BufferedInputStream bis = new
            java.io.BufferedInputStream(new FileInputStream(filename));
        byte[] data = new byte[10*1024];
        int read = bis.read(data);

        pw.println("HTTP/1.0 200 Okay");
        pw.println();
        pw.flush();
        bos.flush();

        while(read != -1)
        {
            bos.write(data,0,read);
            read = bis.read(data);
        }
        bos.flush();
    }
    catch(Exception e)
    {
        pw.flush();
        bos.flush();
    }
}

protected void sendDir(BufferedOutputStream bos,
    PrintWriter pw, File dir, String req) throws Exception
{
    try
    {
        pw.println("HTTP/1.0 200 Okay");
        pw.println();
        pw.flush();

        pw.print("<html><head><title>Directory of ");
        pw.print(req);
        pw.print("</title></head><body><h1>Directory of ");
        pw.print(req);
        pw.println("</h1><table border=\"0\">");

        File[] contents=dir.listFiles();

        for(int i=0;i<contents.length;i++)
        {
            pw.print("<tr>");
            pw.print("<td><a href=\"");
            pw.print(req);
            pw.print(contents[i].getName());
            if(contents[i].isDirectory())
                pw.print("/");
            pw.print("\">");
            if(contents[i].isDirectory())
                pw.print("Dir -> ");
            pw.print(contents[i].getName());
            pw.print("</a></td>");
            pw.println("</tr>");
        }
        pw.println("</table></body></html>");
        pw.flush();
    }
}

```

```

        catch(Exception e)
        {
            pw.flush();
            bos.flush();
        }
    }
}

// Check that a filepath has been specified and a port number
protected void parseParams(String[] args) throws Exception
{
    switch(args.length)
    {
        case 1:
        case 0:
            System.err.println ("Syntax: <jvm> "+
                this.getClass().getName()+"
                docroot port");
            System.exit(0);
        default:
            this.docroot = args[0];
            this.port = Integer.parseInt(args[1]);
            break;
    }
}

public WebServerDemo(String[] args) throws Exception
{
    System.out.println ("Checking for paramaters");

    // Check for command line parameters
    parseParams(args);

    System.out.print ("Starting web server..... ");

    // Create a new server socket
    this.ss = new ServerSocket(this.port);

    System.out.println ("OK");

    for (;;)
    {
        // Accept a new socket connection from our server socket
        Socket accept = ss.accept();

        // Start a new handler instance to process the request
        new Handler(accept, docroot).start();
    }
}

// Start an instance of the web server running
public static void main(String[] args) throws Exception
{
    WebServerDemo webServerDemo = new WebServerDemo(args);
}
}

```

How WebServerDemo Works

The typical Web server must respond to requests from a number of browsers, and will usually handle more than one request for each browser (for example, the parallel downloading of a number of images in an HTML document). This means that such servers need to handle requests for files concurrently, and the simplest method of doing this is to use multiple threads of execution. Those readers who have not yet covered the topic of threads in Java are advised to consult the previous chapter, which examines multi-threading in detail.

This example Web server is designed to be extremely compact and rather plain. It supports the bare minimum of features needed to function as a Web server, namely the HTTP/1.0 `GET` request. No support for dynamic server-side content, such as CGI scripts or Java servlets, is offered—this is a bare-bones Web server example that illustrates how to write a basic server implementation of an RFC, and is intended primarily as a teaching aid. However, it could be used as the scaffolding for a more ambitious project.

For ease of implementation, the server combines all of its code into a single Java source file. This involves the use of an inner class (`Handler`) that deals with the processing of each incoming HTTP request. Most of this server's work is done inside this handler. The reason for this is to give high performance and to handle blocking I/O correctly so that if one client stalls, no others will be affected (since each request handler operates independently).

The outline of the server looks like this:

```
import java.io.*;
import java.net.*;
import java.util.*;

public class WebServerDemo
{
    class Handler extends Thread
    {
    }
}
```

Within the `WebServerDemo` class are stored several variables that are crucial to the server's operation. These are:

- `String docroot`
- `int port`
- `ServerSocket ss`

The `docroot` variable points to a location on a hard drive or network drive where the HTML pages and associated files (such as images) are stored for the Web server to read. The port number is used to track which port the server is operating on. Remember that while port 80 is standard for HTTP, it is common to run on nonstandard ports as well, particularly when an existing server is already running. A popular secondary standard is port 8080. Finally, the `ServerSocket` represents a socket that is bound to a specific port and is listening for incoming client requests. It is from this that we will accept browser requests.

When executed, the `main(String[])` method of the server creates a new server instance. Inside the constructor, the server checks for the presence of the necessary command-line parameters (a filepath for the HTML pages and a port number). This code is handled by the `parseParams(String[])` method, which involves only simple error-checking and string processing. As for the networking code, an attempt is made to bind a server socket to that port, and then the server loops indefinitely while it waits for socket connections.

```
public WebServerDemo(String[] args) throws Exception
```

```

{
    System.out.println ("Checking for paramters");

    // Check for command line parameters
    parseParams(args);

    System.out.print ("Starting web server..... ");

    // Create a new server socket
    this.ss = new ServerSocket(this.port);

    System.out.println ("OK");

    for (;;)
    {
        // Accept a new socket connection from our server@@@
        // socket
        Socket accept = ss.accept();

        // Start a new handler instance to process the
        // request
        new Handler(accept, docroot).start();
    }
}

```

As you can see, the code for accepting a connection and launching a new handler thread to process it is fairly simple. One thing that is readily apparent is that writing a server for one particular protocol is not very different from writing one for another (though, as with everything else in life, there are always exceptions to the rule). The main HTTP-specific work lies in the actual `Handler` class.

Let's examine the `Handler` class in more detail. The handler's constructor is very simple—just enough to store the incoming parameters. We also convert the Web server directory to an absolute location. (Since some operating systems allow directory/drive mappings, it is important to know the actual location, as later we must check for browser attempts to access other, forbidden locations.)

```

public Handler(Socket _socket, String _docroot) throws Exception
{
    socket=_socket;
    // Get the absolute directory of the filepath
    docroot=new File(_docroot).getCanonicalFile();
}

```

When the `Handler` thread is started, the `run()` method will be executed. This is a lengthy method, so we'll break it down into manageable chunks.

As is usual with any networking code, we must wrap it in an exception handler to deal with network errors at runtime. Fortunately, we don't need to do anything about the errors—simply terminating the connection is sufficient. The first step in writing our protocol handler will be to obtain I/O streams and wrap them in a reader/writer.

```

// Prepare our readers and writers
br = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
bos = new BufferedOutputStream(socket.getOutputStream());
pw = new PrintWriter(new OutputStreamWriter(bos));

```

The next step is to read a line of text sent by the browser, which (it is hoped) will contain a valid HTTP command. Of course, under HTTP/1.0, it is possible for commands such as "POST" to be composed of multiple lines, but these commands are not supported by this example. Therefore, we will use the socket's `shutdownInput()` method, which allows for a graceful shutdown of the input reader and instructs the socket to send back acknowledgment of any data coming in and then to silently discard it.

```
// Read HTTP request from user (hopefully GET /file..... )
String line = br.readLine();

// Shutdown any further input
socket.shutdownInput();
```

The next step is to process the HTTP request and work out if the operation is a valid one. We determine first if the line is null and disconnect if it is. As we accept GET requests, we check for a "GET" command, and process the request. If not, the request method sent by the browser is not supported, and we can send back an "HTTP 501 Not Implemented" response. Finally, the output writer and stream are flushed to make sure that the client receives a response.

```
if(line == null)
{
    socket.close();
    return;
}
if(line.toUpperCase().startsWith("GET"))
{
    // Process GET request .....
}
else
{
    pw.println("HTTP/1.0 501 Not Implemented");
    pw.println();
}
pw.flush(); bos.flush();
```

The task of processing the GET request is somewhat more complex, and involves some fancy string processing. While some Java programmers will be comfortable with processing text strings (indeed, Java is an excellent language for this purpose), others may not have come across this before, and may be unaware of the `java.util.StringTokenizer` class. It allows a string to be easily broken into separate pieces, and we use it in this instance to strip off any data (if it exists) after a "?" character. Remember that additional data can be passed to CGI scripts, or to embedded JavaScript, and we must remove it to determine the actual filename that is being requested. Finally, we add the requested filename to our document root, as well as an OS-specific file separator if needed (such as "/" on Wintel systems or "\" on Unix systems).

```
// Eliminate any trailing ? data, such as for a CGI
// GET request
StringTokenizer tokens = new StringTokenizer (line, " ?");
tokens.nextToken();
String req = tokens.nextToken();

// If a path character / or \ is not present, add it
// to the document root
// and then add the file request, to form a full filename
String name;
if(req.startsWith("/") || req.startsWith("\\"))
    name = this.docroot+req;
else
```

```
name = this.docroot+File.separator+req;
```

We then convert this requested file to an absolute filename, and verify that it isn't out of bounds (for example, that it doesn't use a directory mapping on the server to access another drive, or that it doesn't use path operators such as `".."` to look at parent directories). This is one part of a sequence of conditions that must be tested for, such as the existence of a file, if it can be accessed and if it is a directory request instead of a file request. Two methods, `sendDir` and `sendFile`, are used if it is a successful request—otherwise, the appropriate HTTP error status codes are sent to the browser.

```
// Get absolute file path
File file = new File(name).getCanonicalFile();

// Check to see if request doesn't start with our
// document root ....
if(!file.getAbsolutePath().startsWith(this.docroot.getAbsolutePath()))
{
    pw.println("HTTP/1.0 403 Forbidden");
    pw.println();
}
else if(!file.exists())
{
    pw.println("HTTP/1.0 404 File Not Found");
    pw.println();
}
else if(!file.canRead())
{
    pw.println("HTTP/1.0 403 Forbidden");
    pw.println();
}
else if(file.isDirectory())
{
    sendDir(bos,pw,file,req);
}
else // assume this is a file
{
    sendFile(bos, pw, file.getAbsolutePath());
}
```

Breaking the code into two methods (`sendDir` and `sendFile`) simplifies things somewhat, as it leads to easier-to-read code. Since sending a file is pretty much the whole point of HTTP, we'll cover this important method first.

Reading from a file involves the use of a `FileInputStream`. We could, if we positively knew that only text data would be encountered, use a `FileReader`—but often, binary data such as data files and images are sent over HTTP, so we must read and write at the byte level. Starting by sending a valid HTTP 200 response code, we proceed to read the contents of the file and forward it on to the browser. We use the byte array method of reading and writing data for improved efficiency.

```
java.io.BufferedInputStream bis = new
    java.io.BufferedInputStream(new FileInputStream(filename));
byte[] data = new byte[10*1024];
int read = bis.read(data);

pw.println("HTTP/1.0 200 Okay");
pw.println();
pw.flush();
bos.flush();
```



```

while(read != -1)
{
    bos.write(data,0,read);
    read = bis.read(data);
}
bos.flush();

```

The final major method of our HTTP handler that must be discussed is `sendDir`. The task of sending a directory listing, with hyperlinks to files and underlying subdirectories, involves the use of the `File` class, to obtain a list of file entries within a directory. For this method, we will be using a bit of simple HTML, using tables, to create a simple yet readable output.

Now, we start with the HTML. If you're not familiar with raw HTML, don't panic—the code is very brief, and easily understood when examined piece by piece. We start by adding a title to the Web page, which is the directory location that is being requested. Next we create a simple table, which is used to format our data. The directory location is displayed in a large heading (using the `<H1>...</H1>` tags).

```

pw.print("<html><head><title>Directory of ");
pw.print(req);
pw.print("</title></head><body><h1>Directory of ");
pw.print(req);
pw.println("</h1><table border=\"0\">");

```

To obtain a list of files, we use the `File.listFiles()` method, which returns an array of `File` objects. This gives us not only whatever files are stored in our directory, but also the names of any subdirectories. Once obtained, it is a simple process of looping through each element of the array and displaying the output. If the `File` object is actually representing a directory, we also add a `/` to the URL and `Dir ->` to the displayed name of the entry.

```

File[] contents=dir.listFiles();
for(int i=0;i<contents.length;i++)
{
    pw.print("<tr>");
    pw.print("<td><a href=\"");
    pw.print(req);
    pw.print(contents[i].getName());
    if(contents[i].isDirectory())
        pw.print("/");
    pw.print("\">");
    if(contents[i].isDirectory())
        pw.print("Dir -> ");
    pw.print(contents[i].getName());
    pw.print("</a></td>");
    pw.println("</tr>");
}
pw.println("</table></body></html>");
pw.flush();

```

Running WebServerDemo

Unlike other examples, running the server involves a little more complexity. Under some operating systems, such as Unix, many ports are restricted by security settings, or your machine may already have a Web server running. For this reason, you may need to run the server on a nonstandard port (e.g., 8080 rather than 80), although if you're running a plain Wintel system, port 80 should work fine. You must also specify the location of your HTML pages, although for testing you can point it to a directory on your hard drive where you have some images or text documents.

After compiling the Web server, you can run it by typing:

```
java WebServerDemo port docroot
```

where `port` is a TCP port such as 80 or 8080, and `docroot` is a directory.

Next, using your browser, enter the URL `http://hostname:port/`, where "hostname" is the name or IP address of your machine, and "port" is the port you specified when running the server. If you're not connected to a network, you can use "localhost" as the hostname. You should see a directory listing and a series of hyperlinks to files and directories. Click on an image or a Web page, and you'll be able to view them just like a real Web server!

8.4 Summary

The ultimate end of learning networking is for the programmer to write network applications. Theory, while interesting intellectually, cannot replace the experience of actually working with and writing network code. Whether you're writing your own network protocol to suit a special task, or implementing an existing network protocol described by others (such as an RFC), the basics remain the same. Network communication in Java, at its lowest level, takes place over sockets and involves clients and servers.

You're now armed with the knowledge and skill to write your own network applications.

Chapter Highlights

In this chapter, you have learned:

- Where to find specifications of common application protocols (RFCs)
- How to write and run a simple SMTP e-mail client
- How to write and run a simple POP3 e-mail client
- How to write and run a simple HTTP/1.0 server

Chapter 9. HyperText Transfer Protocol

[Section 9.1. Overview](#)

[Section 9.2. HTTP and Java](#)

[Section 9.3. Common Gateway Interface \(CGI\)](#)

[Section 9.4. Summary](#)

9.1 Overview

The HyperText Transfer Protocol (HTTP) is perhaps the most prolific network application protocol in the short history of the Internet. With the possible exception of e-mail, HTTP has changed the face of the Internet more profoundly than any other protocol. Without HTTP, there would be no World Wide Web, no electronic commerce as we know it today, nor the rapid, phenomenal growth of Internet usage.

In previous chapters, we examined HTTP as an application protocol and constructed a simple Web server that serves up Web pages. In this chapter, we'll look at the protocol in greater detail, and how it can be used in Java.

9.1.1 What Is HTTP?

HTTP is an application-level protocol that uses the Transmission Control Protocol (TCP) as a transport mechanism. HTTP provides access to documents and files stored on a Web server. Web browsers use HTTP to request files or dynamically generated content produced by CGI scripts, and other server-side applications. Hypertext documents contain hyperlinks, which are links to other hypertext documents and files. The World Wide Web is a collection of hypertext documents, stored on a wide variety of Web servers and accessed by an even larger number of Web browsers and HTTP clients.

9.1.2 How Does HTTP Work?

When an HTTP client, such as a Web browser or a search engine, needs to access a file, it establishes a TCP connection to the Web server (which, by default, uses TCP port 80 for communication). The client sends a request for a particular file, and receives an HTTP response, which will often include the contents of the file, as shown in [Figure 9.1](#). The response includes a status code (indicating the success or failure of a request), some HTTP header information such as the length of the content and its type, and, if appropriate, the file contents. Each request is for a single file; if subsequent files are needed, additional connections must be made.

Figure 9-1. HyperText Transfer Protocol request/response cycle

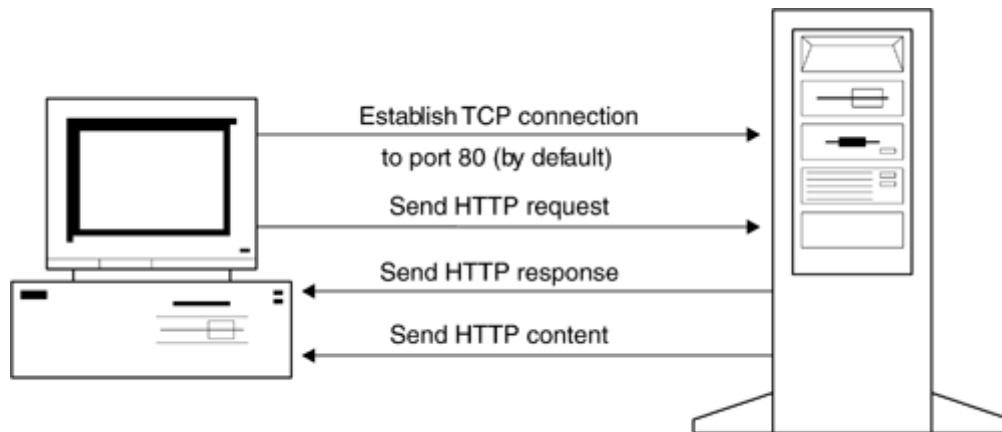


Table 9-1. HTTP Transaction between Client and Server

GET /index.html HTTP/1.0	Client HTTP GET Request
HTTP/1.0 200 OK	←Status line
Last-Modified: Monday, 27-Dec-99 22:14	←Header (composed of several fields)
Content-Type: text/html	GMT
Content-Length: 1639	
<HTML>	←Entity body
<HEAD>	
<TITLE> Our homepage </TITLE>	
</HEAD>	
<BODY>	
..... // BODY GOES HERE	
</BODY>	
</HTML>	

TCP port 80 for communication).The client sends a request for a particular file,and receives an HTTP response,which will often include the contents of the file,as shown in [Figure 9.1](#). The response includes a status code (indicating the success or failure of a request),some HTTP header information such as the length of the content and its type,and,if appropriate,the file contents.Each request is for a single file;if subsequent files are needed,additional connections must be made.

To understand what type of communication takes place between an HTTP client and a Web server, let's look at a sample transaction in which a browser fetches a Web page (see [Table 9-1](#)). Highlighted in bold font is the client request; the server sends the remainder of the transaction.

To fetch the file index.html in the root directory of a Web server, an HTTP client establishes a TCP connection to port 80 and makes an HTTP request. The server processes the request and outputs an HTTP response. Don't be concerned if the meaning of each part of the request and response is not clear to you at this point. The client request and server response are described later in the chapter; for now it is simply important to be aware of what a simple HTTP request/response looks like.

9.1.3 Web Clients

A Web client, connecting to the Web server via a TCP socket, will send an HTTP request and then read back a server response. Earlier versions of HTTP used a simple request format, which would only fetch a resource from the server. Under HTTP/1.0, which most, if not all, servers and clients now support, there are three types of requests that a client application can issue to a Web server:

1. `GET`
2. `HEAD`
3. `POST`

The most common HTTP request is a `GET` request, which fetches a resource from the Web server. More sophisticated requests are possible with `HEAD` and `POST`. Each request can also include header fields, which give the server more information about the client.

9.1.3.1 GET Request Method

When a client needs to retrieve a resource from a Web server, it uses the `GET` request. The `GET` request takes two parameters: the pathname of the resource and the version of HTTP being used. For example:

```
GET /index.html HTTP/1.0
GET /images/banner.gif HTTP/1.0
GET /links.html HTTP/1.0
```

NOTE



CGI parameters can also be passed using the `GET` method. While often only files are requested, it is possible to invoke a CGI script or server-side application.

There are two major variations on the `GET` request, depending on which protocol version of HTTP is in use. The most commonly used version by clients is HTTP/1.0, but servers must maintain backward compatibility with the earlier HTTP/0.9. To issue the older-style request, no HTTP version is specified.

```
GET /index.html
GET /images/adserver.cgi
```

When this style of request is made, no header fields will be returned in the HTTP response. This type of request is rare, and clients should use the HTTP/1.0 version if at all possible.

9.1.3.2 HEAD Request Method

Sometimes a client will be interested in information about a resource but not the resource itself. For example, if a large file is already cached, the client may want to know if it has been modified recently. If so, a new copy would be downloaded, and if not, the cached version would be used. Some clients may be unable to process certain types of content, so they may want to know the MIME content type of the resource. In these situations, a `HEAD` request can be made. The `HEAD` request takes the same parameters as a `GET` request, and will return a normal HTTP response with information about the resource stored in header fields. However, no actual content will be returned, conserving network bandwidth. An example of a `HEAD` request is as follows:

```
HEAD /files/averybigfile.zip HTTP/1.0
```

9.1.3.3 POST Request Method

One of the great advantages of the Web is that Web sites can be interactive. The earliest form of interactivity came in the form of CGI scripts, written in languages such as Perl. CGI is an acronym for the Common Gateway Interface, which defines a standard mechanism for communicating data from browser to server. CGI scripts, and later, server-side applications such as Java servlets (discussed in [Chapter 10](#)) and Active Server Pages (a proprietary Microsoft technology), make it possible for users to interact with server-side applications. However, there must be a way for the browser to pass information from the user to the Web server.

This is where the Common Gateway Interface comes in. CGI parameters are used for this purpose. While it is possible to use the `GET` request to encode such parameters, limitations on the length of a URL are restrictive. A better way is to use the `POST` method, which allows clients to send much more information (including large files) to server-side scripts and applications.

The `POST` request method has a format similar to a `GET` request. After header fields have been sent, however, the client sends an entity body, which contains CGI parameters and other information. When the entity body is complete, the server will process the information and output an HTTP response. [Table 9-2](#) gives a sample transaction, in which a browser submits information from an HTML form using the `POST` request method.

9.1.3.4 Client Request Header Fields

Attached to any request may be header information and a number of optional fields. Header fields are sent after the request line, and are terminated with a carriage return/linefeed. If no headers are specified, the carriage return/linefeed must still be sent.

The HTTP specification defines several header fields that can be used, and clients are free to add their own. However, not every server will honor such fields. If a field is not supported, it will be ignored—no error condition should be generated. The major fields a client is likely to use are covered below.

Table 9-2. POST Request Sending Data to a Server

<code>POST /cgi-bin/information.pl HTTP/1.0</code>	• Client HTTP <code>POST</code> request
<code>Content-type: application/x-www-form-urlencoded</code>	• Header (composed of several fields)
<code>Content-length: 21</code>	
<code>User-Agent: SomeBrowser/1.0</code>	
<code>Name=David%20Reilly&answer=yes</code>	• Entity body
<code>HTTP/1.0 200 OK</code>	← HTTP Server Response
<code>Last-Modified: Monday, 27-Dec-99 22:14 GMT</code>	
<code>Content-Type: text/html</code>	
<code>Content-Length: 4855</code>	
<code><HTML></code>	← Entity body
<code><HEAD></code>	
<code> <TITLE> Thanks for your feedback</code>	
<code></TITLE></code>	
<code></HEAD></code>	
<code><BODY></code>	
<code>..... // BODY GOES HERE</code>	
<code></BODY></code>	
<code></HTML></code>	

"Cookie" Field

Persistent client-side objects (cookies) are small pieces of data sent by a Web server and echoed back on every subsequent request. Cookies are used for tracking client requests and customizing server output for individual users. Not every client supports cookies, and cookie support can be disabled for privacy and security in most clients. An example of this field is:

```
Cookie: secret_id=553235996
```

"From" Field

The "From" field specifies the e-mail address of the user in control of the client. Most Web browsers do not divulge this information (much to the dismay of direct-mail marketers); however, specialized or experimental HTTP clients may. For example, a search engine that sends out requests to remote Web sites may include a contact e-mail address, to help establish contact if the search engine runs wild and requests too many pages in a given period. An example of this field is:

```
From: myemail@mydomain.com
```

"If-Modified-Since" Field

A client that has already requested a resource can use this header field to check whether the resource has been updated. For example, a Web browser that caches pages may send the "If-Modified-Since" field, specifying the date when the resource was last requested. If the resource has been updated, the server will output a normal response and a normal entity body. If, however, it has not been updated, the server will issue a response with a status code of 304 and no entity body. The client then knows to use the cached version. An example of this field is:

```
If-Modified-Since: Tue, 27 Oct 1998, 09:00:00 GMT
```

"Referer" Field

This is an extremely important field for the server, and is often stored for statistical purposes. The "Referer" field specifies the URL that linked to the request URL. If an HTTP client follows a link to a new page, the "Referer" field will specify the page that linked to it. This information is useful for Web masters, as it shows which pages are linking to a site and which search engine queries delivered specific users. Most Web browsers send this field automatically, without the ability to disable it. An example is:

```
Referer: http://www.davidreilly.com/links.html
```

"User-Agent" Field

The "User-Agent" field identifies the type of HTTP client that is making the request. This information can be collected for statistical purposes or used to customize the response issued by the server. For example, an opportunistic Web master might output many random keywords to search engines in an effort to boost the traffic on his or her site. "Real" users would see the original page, since the server would read a browser identification string and not a search engine identification string. Such behavior on the part of a Web master is frowned upon, however, and would eventually result in penalization, or blacklisting of the site.

Developers of HTTP clients should always include this field, to identify the software that is making the request (by default, Java sends a field that identifies HTTP requests as being made by a Java application). Some sites, strange as it may seem, use this information to blacklist certain HTTP clients such as search engines, so it is important that a legitimate "User-Agent" field is sent. By convention, browser applications begin their identification string with the keyword *Mozilla*, which was the identification string of the Netscape browser. An example of this field is:

User-Agent: Mozilla (MyNewBrowser/1.0)

9.1.4 Web Servers

When a client connects to a Web server, it will send an HTTP request that the server will read and process. The server returns a response. That response is made up of the following components:

- A status line, with a numerical status code and human-readable text message
- A response header, with one or more header fields followed by a blank line
- An entity body (optional), which contains the contents of a file or server-side output

Servers should also support the older HTTP/0.9 simple request, for the purpose of backward compatibility. In response to a simple request, a server should return only the entity body and no header information.

9.1.4.1 Status Line

The status line indicates whether or not a request could be performed successfully. If a request could not be completed, it also gives an indication of the reason. It does so by using a three-digit numerical code. The first digit corresponds to an error group, with the two remaining digits indicating a precise error condition.

Each group represents a generic error state (see [Table 9-3](#)). Clients can respond to the first digit (knowing only that an error occurred or that a request was accepted), or they can act based on the type of error that occurred.

Informational (1xx)

The informational set of status codes is rarely used. The HTTP specification states that this code indicates a partial response (such as header information), but with no entity body. These status codes are not a valid response for any HTTP/1.0 request and so should be used only in experimental systems, not for production systems.

Successful (2xx)

This is, under ideal circumstances, the most common set of status codes returned. This indicates that a request was processed and completed without any errors. Although there are several status codes in this group, the most common will be the 200 status code.

Redirection (3xx)

When a resource moves to a new location or a new server, redirection is occasionally used. When a status code from the redirection group is issued, the client should look for a "Location" header field giving the new location of the resource. The entity body of the response may also include a hyperlink to the new resource, in the event that the HTTP client is unable to automatically follow redirect requests.

Table 9-3. Status Codes Used in HTTP

Status Group	Error Group Name
1xx	Informational
2xx	Successful
3xx	Redirection
4xx	Client Error

Table 9-3. Status Codes Used in HTTP

Status Group	Error Group Name
5xx	Server Error

There is also one circumstance in which a redirection status code will be sent, but where no redirection location is specified. If a client issues a conditional request, by specifying an "If-Modified-Since" field in the request header, a 304 status code will be returned, indicating that the resource has not been modified and that the previous cached version should be used. No entity body is returned in this situation.

Client Error (4xx)

When an HTTP client sends an incorrect request, a status code from the client error group will be sent. Reasons for the error can vary, from a bad request, to an invalid URL, to a URL that is forbidden for that client's IP address. The most common of all client error status codes is 404, indicating that the resource was not found. Though this is classed as a client error, sometimes a resource is deleted from the server, making invalid a URL that clients have previously accessed.

Server Error (5xx)

Servers themselves are not impervious to error conditions. When a server is overloaded, for example, it can issue a 503 status code. A server-side CGI script or servlet could malfunction, causing a 501 status code to be issued. Generally, however, servers are reasonably stable—there are more likely to be client errors than server errors.

9.1.4.2 Server Response Header Fields

The response header is composed of two groups of header fields (either a "Response-Header" field, or an "Entity-Header" field). In actual practice, the header group names are irrelevant, as the headers are sent together and only the field name is of interest to a client. Each header field is optional and will not be present in every request. However, some fields will be present in almost all situations, such as the "Content-Type" and "Last-Modified" fields. Covering every possible field would be impossible, as servers can add custom fields. However, the most frequently encountered fields described in the HTTP specification are worth being aware of when writing HTTP clients.

"Location" Field

The location field specifies a URL to a resource. In the case where a redirection status code is specified, the client should fetch the content specified in this header field. For example, in response to a URL request to `http://davidreilly.com/`, a redirection to `http://www.davidreilly.com/` is made using

```
Location: http://www.davidreilly.com/
```

"Server" Field

The "Server" field gives information about the server vendor and server version number. This information often isn't that useful to the client and, as the HTTP specification warns, could represent a security risk. If a server returns this field, and a known security flaw in that vendor/version combination exists, it may open the server to hostile attack. An example of this field is:

```
Server: MyServer v1.05
```

"Content-Length" Field

This field indicates the number of bytes of the entity body. Although not strictly necessary, since the client will stop reading when the entity body terminates, this information can be useful when dealing with large file downloads. At the beginning of the transaction, the client can give the user an estimate of how much content needs to be downloaded, and the time remaining. If a connection terminates prematurely, the client can detect that the file was not completely downloaded, and attempt the request again. An example of this field is:

```
Content-Length: 5934
```

"Content-Type" Field

This field specifies the MIME content type of the entity body. The MIME content type is divided into two sections, separated by a "/" character. The first section indicates the general type of the content (e.g., text, image, application file); the second section indicates the specific type. For example, the content type of a Web page is text/html, whereas a plain text document is text/plain. An image in GIF format is of type "image/gif," whereas a JPEG is of type "image/jpeg." The Internet Assigned Number Authority defines MIME content types, and it is best not to assign arbitrary types in production systems. An example is:

```
Content-Type: image/png
```

"Expires" Field

When a server wishes to prevent caching of a resource for too long a period, it can specify a "use-by" date for a resource. For example, the front page of a news site could specify one hour into the future, to prevent proxy servers (servers that forward HTTP requests on behalf of other machines, such as those trapped within a firewall) and HTTP clients from caching the content longer than an hour. Some content may also be marked as not cacheable, using the "Pragma" header field (see below). However, the "Expires" field allows caching for a specified period of time. An example of this field is:

```
Expires: Thu, 12-Jan-2001 10:00:00 GMT
```

"Last-Modified" Field

The "Last-Modified" field indicates when a resource was last changed or last updated. This information may be useful to proxy servers and clients, to prevent the downloading of large files that are already cached and have not changed since the last request. An example of this field is:

```
Last-Modified: Fri, 22-Feb-1998 15:23:11 GMT
```

"Pragma" Field

Sometimes a server may wish to prevent caching of a resource entirely. For example, a page that changes every few minutes (such as a stock-price information page) or that contains sensitive information that would best not be cached by an intermediary proxy server may be marked with the "Pragma" field. The "Pragma" field can also be used to indicate other restrictions and information on the behavior of clients, though clients are not guaranteed to support such restrictions. An example is:

```
Pragma: no-cache
```

"Set-Cookie" Field

Although not defined in the original HTTP specification, cookies have been quickly adopted by Web browser manufacturers and the Web developer community. When a server-side application wants to send a cookie to the browser, it adds a "Set-Cookie" field to the HTTP response. Subsequent requests by that browser will include the cookie, so that the client can be tracked. Not all browsers have cookies enabled, and server-side applications should be aware that there is no error message sent if a browser does not accept cookies. An example of this field is:

```
Set-Cookie: secret_id=553235996; domain=mysite.com; path=/
```

9.1.4.3 Entity Body

The entity body is the stream of bytes that form the actual content of the requested resource. This content may be static (a file whose content changes infrequently or not at all) or dynamic (a custom server-side response to the client request). Meta-information about the entity body is contained in entity header fields such as "Content-Type" and "Content-Length."

9.2 HTTP and Java

Java provides extremely good support for the HyperText Transfer Protocol. While developers are free to write their own HTTP implementations using TCP sockets, the `java.net` package provides several classes that offer HTTP functionality:

- `java.net.URL`
- `java.net.URLConnection`
- `java.net.HttpURLConnection`

9.2.1 URL Class

The `URL` class represents one of the most frequently used address types of the Internet, the Uniform Resource Locator (URL). URLs can point to files, Web sites, ftp sites, newsgroups, e-mail addresses, and other resources. Some fictitious examples of non-Web URLs are:

- `ftp://records.area51.mil/roswell/subjects/autopsy/`
- `telnet://localhost:8000/`
- `mailto:president@whitehouse.gov?subject=My%20Opinion`

In the context of HTTP and this chapter, we'll be dealing with URLs that point to a Web site, but it is important to remember that other network protocols also use URLs.

The URL is composed of several components (shown in [Figure 9-2](#)), each of which can be parsed by the `URL` class and returned separately. Some components (namely the port and the reference fields) are optional, and will not be present in many URLs. As mentioned earlier, CGI parameters can also be included as part of the path field.

Figure 9-2. Format of the Universal Resource Locator

URL					
protocol	://	hostname	(: port)	path	(# reference)

9.2.1.1 Creating a URL

The `URL` class can be used to parse URLs, or as an identifier of a remote resource that can be employed (in conjunction with the other Java HTTP classes) to retrieve that resource.

Constructors

There are six constructors for the `URL` class; the choice of which to use depends largely on how much control you require over the URL. For most situations, the following constructor will be used:

- `URL ( String url_str )` throws `java.net.MalformedURLException`— creates a `URL` object based on the string parameter. If the URL cannot be correctly parsed, a `MalformedURLException` will be thrown.

However, not every application will have a `String` representation of a URL. For convenience, a wide range of constructors is offered, which some developers may find easier to use.

- `URL ( String protocol, String host, String file )` throws `java.net.MalformedURLException`— creates a `URL` object with the specified protocol, host, and file path.
- `URL ( String protocol, String host, int port, String file )` throws `java.net.MalformedURLException`— creates a `URL` object with the specified protocol, host, port, and file path.
- `URL ( String protocol, String host, int port, String file, URL StreamHandler handler )` throws `java.net.MalformedURLException`— creates a `URL` object with the specified protocol, host, port, file path, and stream handler.
- `URL ( URL context, String relative )` throws `java.net.MalformedURLException`— creates a `URL` object using the context of an existing URL and a relative URL. For example, if we had an existing URL of `http://somewebsite.com/`, and a relative URL of `/images/icon.gif`, the new URL would be `http://somewebsite.com/images/icon.gif`.
- `URL ( URL context, String relative, URLStreamHandler handler )` throws `java.net.MalformedURLException`— creates a `URL` object using the context of an existing URL and a relative URL. Any stream handler specified will override the default stream handler of the context URL.

9.2.1.2 Using a URL

The `URL` class provides the following methods to parse a URL and extract individual components (such as the protocol or the hostname of the URL), as well as to open an HTTP connection to the resource that it specifies.

Methods

- `boolean equals( Object object )`— compares two URLs for equality. If the object is not an instance of the `URL` class, or if the object does not point to an identical resource, a value of "false" is returned.
- `Object getContent()` throws `java.io.IOException`— retrieves the contents of the resource located at the URL. The type of object returned will vary, depending on the MIME content type of the remote resource and the available content handlers (classes responsible for processing and retrieving objects from a `URLConnection`). This method is shorthand for calling the `openConnection()` method, which returns a `URLConnection`, and then invoking the `getContent()` method upon the

`URLConnection` that was returned. As a network connection will be established, an `IOException` may be thrown.

- `String getProtocol()`— returns the protocol component of a URL.
- `String getHost()`— returns the hostname component of a URL.
- `String getPort()`— returns the port component of a URL. This is an optional component, and if not present a value of `-1` will be returned.
- `String getFile()`— returns the pathname component of a URL.
- `String getRef()`— returns the reference component of a URL. This is an optional component, and a reference may not be present. A null value will be returned if no reference was specified.
- `public int hashCode()`— returns an identifier for a `URL` object, for the purpose of hash table indexing.
- `URLConnection openConnection()`— returns a `URLConnection` object, which can be used to establish a connection to the remote resource. The name of this method can be deceiving, though, as no connection will be established until further methods of the `URLConnection` object are invoked.
- `InputStream openStream()` throws `java.io.IOException`— establishes a connection to the remote server where the resource is located, and provides an `InputStream` that can be used to read the resource's contents. This method provides a quick and easy way to retrieve the contents of a URL, without the added complexity of dealing with a `URLConnection` object.
- `boolean sameFile (URL url)`— compares two URLs for equality, similar to that of the `equals (Object)` method. However, only the protocol, hostname, port, and pathname fields are compared—the reference field of the URL is excluded. While the `equals (Object)` method checks that a URL points to the same place in the same file, the `sameFile (URL)` method does not test where in the file the URL points to.
- `String toString()`— returns a `String` representation of a URL. There is no difference between this method and the `toExternalForm()` method.
- `String toExternalForm()`— returns a `String` representation of a URL. There is no difference between this method and the `toString()` method.

9.2.2 Parsing with the URL Class

To demonstrate how the `URL` class is used, and how it can extract the individual components out of a URL, we'll examine a small application that creates an instance of the `URL` class from the command-line parameter passed to it.

Code for URLParser

```
import java.net.*;

// Chapter 9, Listing 1
public class URLParser
{
    public static void main(String args[])
    {
        int argc = args.length;

        // Check for valid number of parameters
        if (argc != 1)
        {
            System.out.println ("Syntax :");
            System.out.println ("java URLParser url");
            return;
        }
    }
}
```

```

        // Catch any thrown exceptions
        try
        {
            // Create an instance of java.net.URL
            java.net.URL myURL = new URL ( args[0] );

            System.out.println ("Protocol : " +
                                myURL.getProtocol() );
            System.out.println ("Hostname : " +
                                myURL.getHost() );
            System.out.println ("Port      : " +
                                myURL.getPort() );
            System.out.println ("Filename : " +
                                myURL.getFile() );
            System.out.println ("Reference: " +
                                myURL.getRef() );
        }
        // MalformedURLException indicates parsing error
        catch (MalformedURLException mue)
        {
            System.err.println ("Unable to parse URL!");
            return;
        }
    }
}

```

How URLParser Works

The example starts by checking that a command-line parameter was passed to the application. A null value for the command-line parameter would cause a runtime exception to be thrown. Next, the application begins a try/catch block, which is required since the URL constructor can throw an instance of `java.net.MalformedURLException`. If an exception is thrown, it will be caught, and a warning sent to the user before terminating.

```

// Catch any thrown exceptions
try
{
    // Create an instance of java.net.URL
    java.net.URL myURL = new URL ( args[0] );

    // Code to process the URL goes here
    .....
}
// MalformedURLException indicates parsing error
catch (MalformedURLException mue)
{
    System.err.println ("Unable to parse URL!");
    return;
}

```

The code to create a `URL` instance is relatively easy. Once you have a `URL`, you can look at the various components of which it is comprised. The application prints out the protocol, host, port, pathname, and reference of the `URL`. Note that the port and reference fields may be blank, so a `-1` and null value, respectively, would be displayed in this case.

```

System.out.println ("Protocol : " + myURL.getProtocol() );
System.out.println ("Hostname : " + myURL.getHost() );
System.out.println ("Port      : " + myURL.getPort() );
System.out.println ("Filename : " + myURL.getFile() );

```

```
System.out.println ("Reference: " + myURL.getRef() );
```

Running URLParser

The `URLParser` application takes a valid URL as its only parameter. A valid URL must contain a protocol, hostname, and pathname components, but it does not necessarily need to point to a valid resource. Thus, the following URLs would be accepted without throwing a

`MalformedURLException`:

- `http://thisisnotarealmachine/noristhisarealpath`
- `http://www.davidreilly.com:80/#top`
- `ftp://ftp.davidreilly.com/pub/`

However, these URLs could not be parsed:

- `abcdef://abcdef.com/alphabetsoup/`
- `ttp://www.davidreilly.com:80/#top`

To run the `URLParser`, specify the URL as the only parameter, as follows:

```
java URLParser url
```

Try running some of the sample URLs, and examine the output for URLs with and without port and reference fields.

9.2.3 Retrieving a Resource with the URL Class

The `URL` class can be used for more than just parsing, however. There are two URL methods that can assist in retrieving the contents of a remote resource:

1. `InputStream URL.openStream()`
2. `URLConnection URL.openConnection();`

For greater control over how the request is made, a `URLConnection` object created by invoking the `URL.openConnection()` method would be used. In many situations, however, a simpler way to retrieve the contents of a resource is called for. The `openStream()` method returns an `InputStream`, which makes reading a resource simple. Examine the following application, which will fetch a URL passed as a command-line parameter.

Code for FetchURL

```
import java.net.*;
import java.io.*;

// Chapter 9, Listing 2
public class FetchURL
{
    public static void main(String args[]) throws Exception
    {
        int argc = args.length;

        // Check for valid number of parameters
        if (argc != 1)
        {
            System.out.println ("Syntax :");
        }
    }
}
```

```

        System.out.println ("java FetchURLConnection url");
        return;
    }

    // Catch any thrown exceptions
    try
    {
        // Create an instance of java.net.URL
        java.net.URL myURL = new URL ( args[0] );

        // Fetch the content, and read from an
        // InputStream
        InputStream in = myURL.openStream();

        // Buffer the stream, for better performance
        BufferedInputStream bufIn = new
            BufferedInputStream(in);

        // Repeat until end of file
        for (;;)
        {
            int data = bufIn.read();

            // Check for EOF
            if (data == -1)
                break;
            else
                System.out.print ( (char) data);
        }

        // Pause for user
        System.out.println ();
        System.out.println ("Hit enter to continue");
        System.in.read();

    }
    // MalformedURLException indicates parsing error
    catch (MalformedURLException mue)
    {
        System.err.println ("Unable to parse URL!");
        return;
    }
    // IOException indicates network or I/O error
    catch (IOException ioe)
    {
        System.err.println ("I/O Error : " + ioe);
        return;
    }
}
}

```

How FetchURL Works

You may notice some similarity between this example and the previous one. Since we're working with the same `URL` class, the code to check for a valid parameter, to construct the `URL`, and catch a possible `MalformedURLException` remains the same. Observant readers may notice an extra catch statement, which handles any network or I/O errors. This is necessary in case the host machine could not be contacted or is invalid. Remember that a `MalformedURLException` will not be thrown if the hostname is invalid, only if the `URL` syntax was not followed.


```
// IOException indicates network or I/O error
catch (IOException ioe)
{
    System.err.println ("I/O Error : " + ioe);
    return;
}
```

Inside the `try/catch` block, the application creates a new `URL` object and calls the `openStream()` method. This establishes a connection to the remote machine (or, if a connection could not be opened, throws an `IOException`). The method returns an `InputStream`, which can then be used for reading from the remote resource.

```
// Create an instance of java.net.URL
java.net.URL myURL = new URL ( args[0] );

// Fetch the content, and read from an InputStream
InputStream in = myURL.openStream();
```

While we could read directly from the input stream, one byte at a time, a more efficient alternative is to connect another stream, one which is better suited to the task at hand. In this case, we use a `BufferedInputStream` and read a byte at a time. However, a `DataInputStream` or a `Reader` object could just as easily have been used.

```
// Buffer the stream, for better performance
BufferedInputStream bufIn = new BufferedInputStream(in);

// Repeat until end of file
for (;;)
{
    int data = bufIn.read();

    // Check for EOF
    if (data == -1)
        break;
    else
        System.out.print ( (char) data);
}

// Pause for user
System.out.println ();
System.out.println ("Hit enter to continue");
System.in.read();
```

While this example is quite simple, it provides only limited control over the connection. For example, what is the MIME content type of the resource? By reading from an `InputStream`, we lose some of the details. The `URLConnection` class, as we'll see in [Section 9.2.4](#), offers greater control over requests and responses.

Running FetchURL

Running the application is straightforward. Simply specify a valid URL as the only command-line parameter, either of a local machine on your network or a Web site, if you are connected to the Internet.

```
java FetchURL url
```

9.2.4 URLConnection Class

The `URLConnection` class is used to send HTTP requests and read HTTP responses. `URLConnection` has methods that allow you to connect to a Web server, to set request header fields, to read response header fields, and of course, to read the contents of the resource.

9.2.4.1 Creating a `URLConnection`

There are no public constructors for the `URLConnection` class. The single constructor for `URLConnection` is marked as a protected method, meaning that only a class in the `java.net` package can create one. Instead, you should call the `URL.openConnection()` method, which will return a `URLConnection` instance.

```
URL url = new URL ( some_url );
URLConnection connection = url.openConnection();
```

9.2.4.2 Using a `URLConnection`

The `URLConnection` provides many methods, the most important of which are listed below. Some are shortcuts to other methods (such as response header fields). For completeness, even advanced methods are included, although for normal programming you would likely use a small subset of these.

Methods

- `void connect()` throws `java.io.IOException`— establishes a connection between the application (the client) and the resource (stored on the server). The act of creating a `URLConnection` object does not establish this connection, and until this method is executed, the remote host will not be contacted. Before invoking this method, the client application should specify any request headers, as these will be sent when the connection is made. Some `URLConnection` methods will call the `connect()` method in order to complete their task.
- `boolean getAllowUserInteraction()`— returns the state of the "allowUserInteraction" field, which indicates whether the URL is being used in a way that supports user interaction, such as providing authentication details.
- `Object getContent()` throws `java.io.IOException`— attempts to process the contents of a resource and return it as an `Object`. The method checks the MIME content type of the resource and attempts to find a suitable content handler.
- `String getContentEncoding()`— returns the value of the "Content-encoding" header field, if such a field exists. Returns null if no content-encoding field was specified.
- `int getContentLength()`— returns the value of the "Content-length" header field, if such a field exists. Returns -1 if no content-length field was specified.
- `String getContentType()`— returns the value of the "Content-type" header field, if such a field exists. Returns null if no content-type field was specified. If no type of information is available, the `guessContentTypeFromStream(InputStream)` method may be used as an indication.
- `long getDate()`— returns the value of the "Date" header field, expressed as the number of seconds since January 1, 1970 GMT. If no such header field was specified, a value of zero will be returned.
- `static boolean getDefaultAllowUserInteractionField()`— returns the default value for the "allowUserInteraction" field. The state of this default value determines the "allowUserInteraction" value of all future `URLConnection` objects that are created.
- `static String getDefaultRequestProperty(String key)`— returns the value of the specified default property, applied to all `URLConnection` objects. Returns null if no such property exists.

- `boolean getDefaultUseCaches()` — returns the default value of the cache flag. The state of this default value determines the cache flag value of all future `URLConnection` objects that are created.
- `boolean getDoInput()` — returns a flag, indicating whether the connection should be used for input. By default, this value will be "true," unless modified by the `setDoInput(boolean)` method.
- `boolean getDoOutput()` — returns a flag indicating whether the connection should be used for output. By default, this value will be "false," unless modified by the `setDoOutput(boolean)` method.
- `long getExpiration()` — returns the value of the "Expires" header field, expressed as the number of seconds since January 1, 1970 GMT. If no such header field was specified, a value of zero will be returned.
- `static FileNameMap getFileNameMap()` — returns an object that implements the `FileNameMap` interface, which is used to map MIME content types to filenames. The content type for a resource may also be determined by calling the `getContentType()` method.
- `String getHeaderField(int n)` — returns the value of the *n*th response header field, or null if this exceeds the number of response header fields available. Unlike an array, which is zero-indexed, the first element of the set of header fields will have an index value of one.
- `String getHeaderField(String field)` — returns the value of the specified header field name, or null if no such field exists.
- `long getHeaderFieldDate (String field, long default_value)` — attempts to parse the specified header field name as a date, expressed as the number of seconds since January 1, 1970 GMT. If a parsing error occurs (such as an invalid date or a missing header field), the `default_value` will be returned.
- `int getHeaderFieldInt (String field, int default_value)` — attempts to parse the specified header field as a number. If a parsing error occurs (such as an invalid number), the `default_value` will be returned.
- `String getHeaderFieldKey(int n)` — returns the name of the *n*th response header field, or null if this exceeds the number of response header fields available. Unlike an array, which is zero-indexed, the first element of the set of header fields will have an index value of one.
- `long getIfModifiedSince()` — returns the date of the "If-Modified-Since" request header field. This field is modified by calling the `setIfModifiedSince(long)` method.
- `InputStream getInputStream()` throws `java.io.IOException`, `java.net.UnknownServiceException` — returns an `InputStream` object that reads the contents of the resource pointed to by the `URLConnection`. If a connection cannot be established, an `IOException` is thrown. In addition, if the connection does not support reading, an `UnknownServiceException` is thrown.
- `long getLastModified()` — returns the date of the "Last-Modified" header field, expressed as the number of seconds since January 1, 1970 GMT. If no such header field was specified, a value of zero will be returned.
- `OutputStream getOutputStream()` throws `java.io.IOException`, `java.net.UnknownServiceException` — returns an `OutputStream` object that writes to the remote connection. If a connection cannot be established, an `IOException` is thrown. In addition, if the connection does not support writing, an `UnknownServiceException` is thrown.
- `Permission getPermission()` throws `java.io.IOException` — returns a `Permission` object, representing the security permissions required to access a resource. If a specialized subclass of `URLConnection` is written, this method should be overridden. Developers won't normally need to use this method, unless special security policies are specified for an application.

- `String getRequestProperty(String key)`— returns the value of the specified property, and null if no such property exists.
- `URL getURL()`— returns a URL object representing the location of the resource pointed to by the `URLConnection`.
- `boolean getUseCaches()`— returns a flag indicating whether or not resources will be cached. If "true," caching will be used wherever possible, and if "false," a new request will be sent each time.
- `protected static String guessContentTypeFromStream(String file)`— returns an estimate of the MIME content type, based on the extension of the specified file.
- `static String guessContentTypeFromStream(InputStream stream)`— returns an estimate of the MIME content type, based on the bytes at the beginning of an `InputStream`. The method does a nondestructive read, in an attempt to determine the content type if header information does not specify it.
- `void setAllowUserInteraction(boolean flag)`— modifies the state of the "allowUserInteraction" field, which indicates whether the URL is being used in a way that supports user interaction, such as providing authentication details.
- `static void setContentHandlerFactory (ContentHandlerFactory factory)` throws `java.lang.SecurityException`, `java.lang.Error`— assigns a content handler factory to all `URLConnection`s. A content handler factory is responsible for creating the appropriate `ContentHandler` based on the MIME content type of the resource. If a content handler already exists, or if the current security manager prohibits the operation, an error will occur.
- `static void setDefaultAllowUserInteraction(boolean flag)`— modifies the default value of the state of the "allowUserInteraction" field, which will be assigned to all future `URLConnection` objects.
- `static void setDefaultRequestProperty(String key)`— modifies the value of the specified default property applied to all `URLConnection` objects.
- `void setDefaultUseCaches(boolean flag)`— modifies the default value of the cache flag. The state of this default value determines the cache flag value of all future `URLConnection` objects that are created.
- `void setDoInput()`— modifies the input flag, indicating whether the connection should be used for reading.
- `void setDoOutput()`— modifies the output flag, indicating whether the connection should be used for writing.
- `void setIfModifiedSince(long date)`— sets the "If-Modified-Since" request header, indicating that a resource should only be fetched if modified after a certain date. This method must be invoked prior to the `connect()` method if the setting is to take effect.
- `void setRequestProperty(String key, String value)`— assigns the specified value to the specified request header field. This method must be invoked prior to the `connect()` method if the setting is to take effect.
- `void setUseCaches(boolean flag)`— modifies the flag indicating whether caching of requests should be allowed. If "true," caching will be used wherever possible, and if "false," a new request will be sent each time.

9.2.5 Retrieving a Resource with the `URLConnection` Class

While the `URL` class does allow you to retrieve a resource by using the `URL.openConnection()` method, information about the resource is lost, as is the ability to prevent caching of requests and to specify additional header fields, since only a stream object is returned. The example below shows how to use the `URLConnection` class to retrieve a resource and to determine its MIME content type and the length of the resource. Since we still use the `URL` class, some of the code is similar to previous examples.

Code for FetchURLConnection

```
import java.net.*;
import java.io.*;

// Chapter 9, Listing 3
public class FetchURLConnection
{
    public static void main(String args[]) throws Exception
    {
        int argc = args.length;

        // Check for valid number of parameters
        if (argc != 1)
        {
            System.out.println ("Syntax :");
            System.out.println ("java FetchURLConnection url");
            return;
        }
        // Catch any thrown exceptions
        try
        {
            // Create an instance of java.net.URL
            java.net.URL myURL = new URL ( args[0] );

            // Create a URLConnection object, for this URL
            // NOTE : no connection has yet been
            // established
            URLConnection connection = myURL.openConnection();

            // Now open a connection
            connection.connect();

            // Display the MIME content-type of the
            // resource (e.g. text/html)
            String MIME = connection.getContentType();
            System.out.println ("Content-type: " + MIME);

            // Display, if available, the content length
            int contentLength = connection.getContentLength();
            if (contentLength != -1)
            {
                System.out.println ("Content-length: " +
contentLength);
            }

            // Pause for user
            System.out.println ("Hit enter to continue");
            System.in.read();

            // Read the contents of the resource from the
            // connection
            InputStream in = connection.getInputStream();

            // Buffer the stream, for better performance
            BufferedInputStream bufIn = new
BufferedInputStream(in);

            // Repeat until end of file
            for (;;)
            {
```

```

        int data = bufIn.read();

        // Check for EOF
        if (data == -1)
            break;
        else
            System.out.print ( (char) data);
    }
}
// MalformedURLException indicates parsing error
catch (MalformedURLException mue)
{
    System.err.println ("Unable to parse URL!");
    return;
}
// IOException indicates network or I/O error
catch (IOException ioe)
{
    System.err.println ("I/O Error : " + ioe);
    return;
}
}
}

```

How FetchURLConnection Works

We start by checking for a valid parameter, and create a `URL` instance, as in previous examples. Rather than calling `URL.openStream()` once we have a `URL` object, however, the `openConnection()` method is called instead, returning a `URLConnection` instance.

```

// Create an instance of java.net.URL
java.net.URL myURL = new URL ( args[0] );

// Create a URLConnection object, for this URL
// NOTE : no connection has yet been established
URLConnection connection = myURL.openConnection();

```

The name of the `openConnection()` method is somewhat misleading. Although it creates an instance of the `URLConnection` object, it does not establish an HTTP session with the Web server. This can be advantageous, as it allows us to set any request header fields we need. For a simple example like this, however, this functionality is not needed, so we can connect to the Web server immediately, which sends a request for the `URL` associated with this connection.

```

// Now open a connection
connection.connect();

```

Once the connection is established and the request sent, a response will be issued by the Web server. This response will include a variety of header fields, as discussed earlier in the chapter. The most important field of all is the "Content-Type," which tells an application whether the resource is text, an image, a data file, or some other resource. The application could read this data by calling the `URLConnection.getHeaderField(String)` method and passing a value of "Content-Type." However, a shortcut method exists that makes for more readable source code.

```

// Display the MIME content-type of the resource (e.g. text/html)
String MIME = connection.getContentType();
System.out.println ("Content-type: " + MIME);

```

Another important piece of information is the length of the resource. Large files can take minutes or even hours to download, and the user benefits from knowing the length of the resource at the beginning of the transaction. This information is provided in the "Content-Length" header field, and a shortcut method exists for this data that converts it to an int value. After displaying the length, the application pauses, to allow the user time to read it before displaying the requested resource.

```
// Display, if available, the content length
int contentLength = connection.getContentLength();
if (contentLength != -1)
{
    System.out.println ("Content-length: " + contentLength);
}

// Pause for user
System.out.println ("Hit enter to continue");
System.in.read();
```

The next step is to get an `InputStream` to the contents of the resource. Just like a `URL` object, `URLConnection` provides a method to create an `InputStream` for reading a resource. For this purpose, the `URLConnection.getInputStream()` method is used.

```
// Read the contents of the resource from the connection
InputStream in = connection.getInputStream();
```

Once an `InputStream` has been obtained, the resource is read in the same way as the previous example. The chief difference between `FetchURL` and `FetchURLConnection` is that more information about the resource was provided. The `URLConnection` class can be used for more than just content type and length information, however. In the next examples, we'll unleash the power of the `URLConnection` class.

Running FetchURLConnection

This application takes the same command-line parameter as the previous example. Specify a valid URL as the only command-line parameter, either of a local machine on your network, or a Web site, if you are connected to the Internet.

```
java FetchURLConnection url
```

The application will issue an HTTP request, then display the MIME content type and length of the resource and then the resource itself.

9.2.6 Modifying and Examining Header Fields with URLConnection

In the previous example, you learned how to use the `URLConnection` to fetch HTTP resources, and how to determine the length and MIME content type of a resource. However, there are many more HTTP response header fields to examine, and you can also modify HTTP request header fields to make it possible for server-side applications (such as CGI scripts, Active Server Pages, or Java servlets) to customize their output. In this next example, you'll learn how to modify request properties, and how to get response header fields.

Code for HTTPHeaders

```
import java.net.*;
import java.io.*;
```

```

// Chapter 9, Listing 4
public class HTTPHeaders
{
    public static void main(String args[]) throws Exception
    {
        int argc = args.length;

        // Check for valid number of parameters
        if (argc != 1)
        {
            System.out.println ("Syntax :");
            System.out.println ("java HTTPHeaders url");
            return;
        }

        // Catch any thrown exceptions
        try
        {
            // Create an instance of java.net.URL
            java.net.URL myURL = new URL ( args[0] );

            // Create a URLConnection object, for this URL
            // NOTE : no connection has yet been established
            URLConnection connection = myURL.openConnection();

            // Set some basic request fields
            // Set user agent, to identify the application
            // as Netscape compatible
            connection.setRequestProperty ("User-Agent",
                "Mozilla/4.0 (compatible; JavaApp)");

            // Set our referer field - set to any URL you'd
            // like
            connection.setRequestProperty ("Referer",
                "http://www.davidreilly.com/");

            // Set use-caches field, to prevent caching
            connection.setUseCaches(false);

            // Now open a connection
            connection.connect();

            // Examine request properties, to verify their
            // settings
            System.out.println ("Request properties....");
            System.out.println();
            System.out.println ("User-Agent: " +
                connection.getRequestProperty("User-Agent"));
            System.out.println ("Referer: " +
                connection.getRequestProperty("Referer"));
            System.out.println (); System.out.println ();

            // Examine response properties, to see their
            // settings
            System.out.println ("Response properties....");
            System.out.println();

            int i = 1;

            // Search through each header field, until no
            // more exist

```



```

        while ( connection.getHeaderField ( i ) != null )
        {
            // Get the name of this header field
            String headerName =
connection.getHeaderFieldKey(i);

            // Get the value of this header field
            String headerValue =
connection.getHeaderField(i);

            // Output header field key, and header
            //field value
            System.out.println ( headerName + ": " +
headerValue);

            // Goto the next element in the set of
            // header fields
            i++;
        }
        // Pause for user
        System.out.println ("Hit enter to continue");
        System.in.read();
    }
    // MalformedURLException indicates parsing error
    catch (MalformedURLException mue)
    {
        System.err.println ("Unable to parse URL!");
        return;
    }
    // IOException indicates network or I/O error
    catch (IOException ioe)
    {
        System.err.println ("I/O Error : " + ioe);
        return;
    }
}
}

```

How HTTPHeaders Works

Like previous examples, this example uses an instance of the `URLConnection` class to issue HTTP requests. The chief difference here is that, before any request is sent, custom HTTP request fields are added. These header fields provide additional information to server-side applications, which can then be used to customize the HTTP response. When a Web browser sends a request, it identifies itself by sending a "User-Agent" field in the request. Well-behaved HTTP clients do the same, and it is often advantageous to pose as a Web browser by including the *Mozilla* keyword in the identification string and then appending a legitimate-sounding application name, since CGI scripts and servlets sometimes offer different output depending on whether it's an HTTP agent like a search engine or an actual browser. Other request fields can also be set, such as the referring URL and the cache flag, which determines whether or not a unique request will be sent each time to the server.

```

// Create a URLConnection object, for this URL
// NOTE : no connection has yet been established
URLConnection connection = myURL.openConnection();

// Set some basic request fields

// Set user agent, to identify the application as Netscape compatible
connection.setRequestProperty ("User-Agent",

```

```

    "Mozilla/4.0 (compatible; JavaApp)");
// Set our referer field
connection.setRequestProperty ("Referer",
"http://www.davidreilly.com/");

// Set use-caches field, to prevent caching
connection.setUseCaches(false);

```

Once the request settings are made, the `URLConnection` object can send the request. If a call to the `connect()` method is made before assigning request properties, then the server will not receive them and they will not take effect. After connecting and sending the request, the application displays the request fields to the user.

```

// Now open a connection
connection.connect();

// Examine request properties, to verify their settings
System.out.println ("Request properties....");
System.out.println();
System.out.println ("User-Agent: " +
connection.getRequestProperty("User-Agent"));
System.out.println ("Referer: " +
connection.getRequestProperty("Referer"));
System.out.println (); System.out.println ();

```

The next set of header fields displayed by the application is from the server response. In the previous section, the names of the request fields were known. It would be impossible, however, to know the name of every field that might be sent back by a server. Not all servers support the same fields, and some server-side applications may send back custom fields that a client has never before encountered. The `URLConnection` offers several methods that provide access to request fields, two of which support a numerical index value rather than a key name. This allows us to read the *n*th key, and to iterate through every element in the set of header fields. The program prints out both the name of the field and its contents.

```

// Examine response properties, to see their settings
System.out.println ("Response properties....");
System.out.println();

int i = 1;

// Search through each header field, until no more exist
while ( connection.getHeaderField ( i ) != null )
{
    // Get the name of this header field
    String headerName = connection.getHeaderFieldKey(i);

    // Get the name of this header field
    String headerValue = connection.getHeaderField(i);
    // Output header field key, and header field value
    System.out.println ( headerName + ": " + headerValue);

    // Goto the next element in the set of header fields
    i++;
}

```

Finally, the application pauses to allow the user to read the header fields, and then terminates. The actual contents of the resource are not displayed, although code for this exists in the previous example.

Running HTTPHeaders

This application takes the same command-line parameter as previous examples. You should pass a valid URL as the only command-line parameter, either of a local machine on your network, or a Web site, if you are connected to the Internet. Try URLs pointing to different Web servers, to see a variation in the type of headers returned.

```
java HTTPHeaders url
```

The application will issue an HTTP request, display all request and response headers, and then terminate.

9.2.7 HttpURLConnection Class

One of the problems of reading resources using `URL.openStream()` or the `URLConnection` class is that access to HTTP-specific functionality is not available. Any protocol for which a registered protocol handler exists can be fetched in this manner, including the File Transfer Protocol (FTP). But there is no notion of a request method, or a response status code, in these other protocols. How does an application know whether a resource was found, or a 404 "Not Found" error message was sent?

Indeed, many servers output custom pages, designed more for end users than for HTTP client applications. However, there is no uniform error message placed in the message body that is standard across all Web servers. The solution is to read the response status code (discussed earlier in this chapter), as it is the only appropriate way to determine the success or failure of requests.

Though support for HTTP-specific functionality did not exist in earlier versions of Java, as of JDK1.1 there is a solution in the form of the `HttpURLConnection` class. This class extends the `URLConnection` class, and provides additional methods and fields that encapsulate HTTP functionality.

9.2.7.1 Creating a HttpURLConnection

There are no public constructors for the `HttpURLConnection` class, just as in the case of `URLConnection`. The single constructor for `HttpURLConnection` is marked as a protected method, meaning that only a class in the `java.net` package can create one. Instead, you should call the `URL.openConnection()` method, which will return a `URLConnection` instance. The `URLConnection` class is the superclass of `HttpURLConnection`, and if the protocol field of the URL is set to HTTP, this method will actually return an `HttpURLConnection` instance. To gain access to HTTP-specific functionality, you should test to see whether or not the object is an instance of `HttpURLConnection`; if so, the object must be cast as such.

```
URL url = new URL ( some_url );
URLConnection connection = url.openConnection();

if (connection instanceof java.net.HttpURLConnection)
{
    HttpURLConnection httpConnection = (HttpURLConnection) connection;

    // do something with httpConnection
}
```

NOTE



Even if an `HttpURLConnection` object is expected, it is good programming practice to test the class type using the *instanceof* keyword. If an unexpected class is returned by the `URL.openConnection()` method (for example, if a different protocol to HTTP was specified by the URL), a runtime error will occur and the application may not handle it gracefully.

9.2.7.2 Using an `HttpURLConnection`

The `HttpURLConnection` class inherits all of the functionality (including fields and methods) of its parent class, `URLConnection`. It also adds additional functionality, in the form of methods that allow greater access to HTTP features, and static fields that represent common HTTP states.

Fields

The `HttpURLConnection` class defines many static fields, which represent HTTP status codes. While an application can refer to a status code by a numerical value, these fields may make for more readable code. The following fields are all public static final int fields. (To conserve space, the public static final access identifiers have been omitted.)

- `int HTTP_OK`— HTTP status code (200) indicating that the request was successful.
- `int HTTP_CREATED`— HTTP status code (201) indicating that a resource was created.
- `int HTTP_ACCEPTED`— HTTP status code (202) indicating that the request was accepted but has not yet been acted upon.
- `int HTTP_NOT_AUTHORITATIVE`— HTTP status code (203) indicating that the set of header fields is not from the original source. This may indicate that it has been cached locally or read from a third-party copy, and may represent a subset or superset of fields.
- `int HTTP_NO_CONTENT`— HTTP status code (204) indicating that an entity body was not available. For example, if data was sent to a server-side application but no entity body was needed, this code might be returned.
- `int HTTP_RESET`— HTTP status code (205) indicating that the browser should reset the view of the document, but that no new document is available. For example, an HTML form should be cleared, to allow new data to be input by the user.
- `int HTTP_PARTIAL`— HTTP status code (206) indicating that the Web server was able to fulfill a partial `GET` request for a resource. Partial requests occur when an HTTP client has part, but not all, of a resource, and wishes to request only the missing data. If a resource is modified, the new resource will be sent and this status code will not be issued.
- `int HTTP_MULT_CHOICE`— HTTP status code (300) indicating that the resource can be found at multiple locations, from which the client can choose. When a resource is located elsewhere, a "Location" entity field will be sent, along with a 3xx redirection status code, but in the case of multiple choices of location, this status code will be issued.
- `int HTTP_MOVED_PERM`— HTTP status code (301) indicating that the location of a resource has moved permanently and the client should look for a "Location" field in the HTTP response. The new location of the resource should be used in future.
- `int HTTP_MOVED_TEMP`— HTTP status code (302) indicating that a temporary change has been made to the location of the resource, indicated by a "Location" field in the HTTP response.
- `int HTTP_SEE_OTHER`— HTTP status code (303) indicating that a `GET` request should be used to fetch the resource, at a location specified by the "Location" field. This is often issued in response to a `POST` request, which processes the information and redirects to a standard page.
- `int HTTP_NOT_MODIFIED`— HTTP status code (304) used to inform the client that a resource has not been modified and that no entity body was sent. This is used in

conjunction with the "If-Modified-Since" request field, which performs a conditional `GET` request.

- `int HTTP_USE_PROXY`— HTTP status code (305) that informs the client that a proxy server (a server that makes requests on behalf of clients, usually ones trapped within a firewall) must be used to access this resource. The "Location" field indicates the location of the proxy server, which should be used to reissue the request.
- `int HTTP_BAD_REQUEST`— HTTP status code (400) that is issued in response to an invalid HTTP request, which fails to follow the correct syntax.
- `int HTTP_UNAUTHORIZED`— HTTP status code (401) indicating that access to the resource requires user authentication.
- `int HTTP_PAYMENT_REQUIRED`— HTTP status code (402) used to indicate that payment is required for access to this resource. This status code is reserved for the future, and is not in common use.
- `int HTTP_FORBIDDEN`— HTTP status code (403) indicating that access to a resource is strictly forbidden.
- `int HTTP_NOT_FOUND`— HTTP status code (404) used to notify a client that the resource could not be found or has been permanently removed.
- `int HTTP_BAD_METHOD`— HTTP status code (405) that tells the client that the request method is not supported by the server. A list of allowed methods will be specified in the "Allow" response header field.
- `int HTTP_NOT_ACCEPTABLE`— HTTP status code (406) that notifies the client that the response contains content with attributes violating those prescribed in the request.
- `int HTTP_PROXY_AUTH`— HTTP status code (407) indicating that the client must authenticate itself to the proxy. This is a slight, but important, difference between that of status code 401.
- `int HTTP_CLIENT_TIMEOUT`— HTTP status code (408) that occurs when a client fails to send a request within the required timeframe set by the server.
- `int HTTP_CONFLICT`— HTTP status code (409) that notifies the client that access to a resource is temporarily unavailable but may be available at a later date. For example, an access conflict might occur if a resource is being modified, and a request to read that resource occurs.
- `int HTTP_GONE`— HTTP status code (410) that indicates that a resource has been permanently removed. This is similar to the 404 status code, but indicates that the resource is gone forever.
- `int HTTP_LENGTH_REQUIRED`— HTTP status code (411) issued when a client fails to specify a required "Content-Length" field.
- `int HTTP_PRECON_FAILED`— HTTP status code (412) indicating that a precondition placed on the request evaluated to false, and the request could not proceed.
- `int HTTP_ENTITY_TOO_LARGE`— HTTP status code (413) indicating that the request entity was too large to process.
- `int HTTP_REQ_TOO_LONG`— HTTP status code (414) indicating that the request path was too long to process.
- `int HTTP_UNSUPPORTED_TYPE`— HTTP status code (415) notifying the client that the format of the request entity was unsupported.
- `int HTTP_SERVER_ERROR`— HTTP status code (500) indicating that a server error occurred and the request could not be processed.
- `int HTTP_INTERNAL_ERROR`— HTTP status code (501) indicating that a server did not know how to perform the request.
- `int HTTP_BAD_GATEWAY`— HTTP status code (502) indicating that an error occurred while acting as a gateway or proxy server.
- `int HTTP_UNAVAILABLE`— HTTP status code (503) indicating that the server could not process the request due to a temporary condition such as a server overload.
- `int HTTP_GATEWAY_TIMEOUT`— HTTP status code (504) indicating that the server, while acting as a gateway or proxy, did not receive a response in time from another server.

- `int HTTP_VERSION`— HTTP status code (505) indicating that the server did not support that HTTP version of the request.

Methods

New methods added by `URLConnection` are listed below. To conserve space, we have not listed here methods inherited from the `URLConnection` class.

- `void disconnect()`— if a connection to the Web server is still active, the connection is closed.
- `InputStream getErrorStream()`— returns an `InputStream` instance that can be used to read error messages sent by the server. If a connection has not yet been established, or no errors have yet occurred, this method returns null.
- `static boolean getFollowRedirects()`— indicates whether HTTP redirects will be automatically followed. Returns "true" if redirection will occur automatically, and "false" if not.
- `String getRequestMethod()`— returns the request method (e.g., `GET`) being used.
- `int getResponseCode()`— returns the response status code. Applications can hardwire the numerical value of codes, or use the `URLConnection` fields that define state conditions.
- `String getResponseMessage()`— returns the message from the response status line, such as "OK," or "Not Found."
- `static void setFollowRedirects(boolean flag)` throws `java.lang.SecurityException`— determines whether the resource specified in a redirection response will be automatically followed. This must be invoked prior to the `connect()` method for the setting to take effect. If this violates the settings of the security manager, a `SecurityException` will be thrown.
- `void setRequestMethod(String method)` throws `java.net.ProtocolException`— sets the request method for this connection. This must be invoked prior to the `connect()` method for the setting to take effect. If the method is not supported, a `ProtocolException` will be thrown. The method name must be capitalized, as the protocol names are case sensitive.
- `boolean usingProxy()`— shows whether a proxy server is being used for this connection. Returns "true" if using a proxy server, "false" if not.

9.2.8 Accessing HTTP-Specific Functionality Using `URLConnection`

This next example shows how to read the status code and message of a response. Most URLs will generate a "normal" response, with a 200 status code and a normal entity body. However, in some situations (for example, if a resource has moved or if the request method is not supported) this will not be the case.

Code for Using `URLConnection`

```
import java.net.*;
import java.io.*;

// Chapter 9, Listing 5
public class UsingURLConnection
{
    public static void main(String args[]) throws Exception
    {
        int argc = args.length;
```

```

// Check for valid number of parameters
if (argc != 1)
{
    System.out.println ("Syntax :");
    System.out.println ("java UsingURLConnection url");
    return;
}

// Catch any thrown exceptions

try
{
    // Create an instance of java.net.URL
    java.net.URL myURL = new URL ( args[0] );

    // Create a URLConnection object, for this URL
    // NOTE : no connection has yet been established
    URLConnection connection = myURL.openConnection();

    // Check to see if connection is a HttpURLConnection
instance
    if (connection instanceof java.net.HttpURLConnection)
    {
        // Yes... cast to a HttpURLConnection instance
        HttpURLConnection hConnection =
            (HttpURLConnection) connection;

        // Disable automatic redirection, to see the
status header
        hConnection.setFollowRedirects(false);

        // Connect to server
        hConnection.connect();

        // Check to see if a proxy server is being used
        if (hConnection.useProxy())
        {
            System.out.println
                ("Proxy server used to access resource");
        }
        else
        {
            System.out.println
                ("No proxy server used to access resource");
        }

        // Get the status code
        int code = hConnection.getResponseCode();

        // Get the status message
        String msg = hConnection.getResponseMessage();

        // If a 'normal' response
        if ( code == HttpURLConnection.HTTP_OK )
        {
            // Notify user
            System.out.println ("Normal response
returned : " +
                code + " " + msg );
        }
        else

```

```

        {
            // Output status code and message
            System.out.println ("Abnormal response
returned : "
                                + code + " " + msg );
        }

        // Pause for user
        System.out.println ("Hit enter to continue");
        System.in.read();
    }
    else
    {
        System.err.println ("Invalid transport protocol -
not http!");
        return;
    }
}
// MalformedURLException indicates parsing error
catch (MalformedURLException mue)
{
    System.err.println ("Unable to parse URL!");
    return;
}
// IOException indicates network or I/O error
catch (IOException ioe)
{
    System.err.println ("I/O Error : " + ioe);
    return;
}
}
}

```

How UsingHttpURLConnection Works

The program starts by creating a `URL` object, and from this, a `URLConnection` object. If the protocol being used to request the resource is HTTP, then the `URLConnection` will also be an instance of the `HttpURLConnection` class. It is not advisable to presume that this will always be the case, however. A guard statement checks to see if it is an `HttpURLConnection` object and performs a casting operation. If not, an error message is displayed and the program terminates.

```

// Create a URLConnection object, for this URL
// NOTE : no connection has yet been established
URLConnection connection = myURL.openConnection();

// Check to see if connection is a HttpURLConnection instance
if (connection instanceof java.net.HttpURLConnection)
{
    // Yes... cast to a HttpURLConnection instance
    HttpURLConnection hConnection = (HttpURLConnection) connection;

    // Do something with hConnection
    ...
}
else
{
    System.err.println ("Invalid transport protocol - not http!");
    return;
}

```


If all proceeds according to plan, the application now has an `HttpURLConnection`, and the extra HTTP-specific functionality it gives. Before a connection is established, it is possible to modify the properties of the request. For example, a different request method could be used, or the "follow redirection" flag could be modified. So that users can see the redirection status code, automatic redirection is disabled by the application, and then the connection is established.

```
// Disable automatic redirection, to see the status header
hConnection.setFollowRedirects(false);

// Connect to server
hConnection.connect();
```

Once a connection has been established, all sorts of useful information becomes available, such as the status code and message and whether or not a proxy server is being used. The application checks for the presence of a proxy server, and displays it to the user, by using the boolean `HttpURLConnection.usingProxy()` method.

```
// Check to see if a proxy server is being used
if (hConnection.usingProxy())
{
    System.out.println ("Proxy server used to access resource");
}
else
{
    System.out.println ("No proxy server used to access resource");
}
```

Next, the status code and message are retrieved. This is the most useful information of all, as it tells a client whether or not a request was successful and, if it was not successful, gives an indication of why. The human-readable shortcut for the 200 status code (`HttpURLConnection.HTTP_OK`) is used to check whether the request was successful. If not, the status code and message are displayed to the user, and an application could take further steps, such as resending a request or following a redirection notice.

```
// Get the status code
int code = hConnection.getResponseCode();

// Get the status message
String msg = hConnection.getResponseMessage();

// If a 'normal' response
if ( code == HttpURLConnection.HTTP_OK )
{
    // Notify user
    System.out.println ("Normal response returned : " + code + " " +
msg );
}
else
{
    // Output status code and message
    System.out.println ("Abnormal response returned : " + code + " "
+ msg );
}
```

Running UsingHttpURLConnection

The `UsingHttpURLConnection` application accepts as a single parameter the URL of the resource you want to investigate. The syntax is as follows: